VASTRANGAM

Vastrangam — Build Roadmap

Everything, in one file: the ten stages from idea to launch, then all 22 modules, all 113 apps and all 293 rules in full — each rule with what the system does, what it refuses to do instead, and the test that proves it where one exists.

What this document says about what exists

Every other document here describes a design and carries no build-state labels, deliberately: where every line would say the same thing, a label is noise a reader mistakes for information. **This document is the exception.** It is a roadmap, so what is standing up and what is written down is the thing you came to find out — and the label genuinely varies now.

There are three different ways an app can be real here, and they are not interchangeable:

State	Meaning	Count
RUNNING	On the real database, inside row-level security, with a test that starts it and drives it	3
BROWSER APP	Opens in a browser and carries its own self-tests. No shared database behind it	16
ENGINE	The arithmetic is written and passes on the command line. No screen	2
SPECIFIED	Designed and ruled, not built	94

One app is counted twice above — a browser screen came first and the platform implementation second, and both are true. 113 apps in total.

	Count	
Modules	22	one of them is the spine, not a screen you open
Apps	113	3 running, 16 browser apps, 2 engines
Rules	293	**89 proven by a test that runs**, 204 specified
Database tables	151	executing into PostgreSQL, isolation enforced by the database
Stack layers	19	57 named alternatives between them
Specified screens	22	column by column

The gap between 89 and 293 is the build queue. It is not a rounding error and it is not hidden: every rule below says which side of it it is on.

Part one — from idea to launch

10 stages. Each says what it is, what it owes before it can be called finished, how you know it is, and — where something is genuinely missing — what is not covered yet.

Stage	Owes	Anything missing
Idea	4	no
Product architecture	4	no
Design	4	no
Development	4	no
Infrastructure	4	no
Security	4	no
Testing	4	no
Deployment	4	no
Monitoring	4	yes — stated in full below
Launch	4	yes — stated in full below

1 · Idea

A business does not buy software. It buys the end of a particular daily aggravation, and everything else is overhead it agrees to carry. So the idea is stated as the aggravation, not as the product: a manufacturer selling through a shop, a website, several marketplaces and a wholesale book keeps that business alive in a dozen spreadsheets and two WhatsApp groups, and no two of them agree. Stock is a number somebody remembers. The tax return is assembled by hand from files that were themselves assembled by hand. Nobody can say what a finished item cost to make without an afternoon of work, and by then the answer has moved.

The system is one place where each of those facts is written once. Not a better spreadsheet — a spreadsheet is a place to put a number, and the number is the easy part. What is hard is the RULE around the number: who may change it, what happens to last month’s books when they do, and what the system refuses to do when the number is wrong.

The product is that engine, sold to many businesses. A trade is a row of configuration rather than a version of the software, because the moment one customer gets a branch in the code the product has started to split, and in two years there are as many versions as there are customers, each needing its own fix for the same bug.

What this stage owes

- The aggravation named in the customer’s own words, not the vendor’s
- What the business does today instead, and what that costs it
- The one thing that must be true for the product to be worth buying at all
- Who it is NOT for — a product for everybody is a product with no rules

How you know it is done. Somebody who runs a business of this kind reads the first page and recognises their own week in it. That is not a test a machine can run, and this document does not pretend otherwise.

---

2 · Product architecture

The shape of the thing, and the arguments for that shape. One database with the isolation enforced by the database itself rather than by application code; one set of tables that every trade shares; configuration as data rather than as forks; money as whole paise because a decimal loses a rupee somewhere and never says where.

The part worth reading is not the decision but what would make it wrong. A design argued only in its own favour is a sales document. Every decision in the architect register carries a "wrong if" — the condition under which it should be revisited — and the register’s own checker refuses a section that has none.

What this stage owes

- Every structural decision, with what would make it wrong
- The layers, what each is built on, and what could replace it
- What is shared between businesses and what is theirs alone
- The guarantees a trade configuration may never switch off

How you know it is done. node brand/site/checkstack.js --summary — every layer has a default, at least two named alternatives and an interface. No capability may depend on one product.

Where this is written down: `brand/site/architect.js` · `brand/site/stack.js` · `brand/site/dynamic.js`

---

3 · Design

What a person actually sees, column by column. A screen specification here is not a picture — it is the columns on the table, the rows that appear in it, and the buttons that act on them, written down so the built screen can be compared against something.

The screens are specified per module rather than per app, and there are fewer of them than there are apps. That is stated rather than smoothed over: a module with a screen has one worked example of what its apps look like, and a module without one has a description and no picture.

What this stage owes

- The columns, rows and controls of each specified screen
- One renderer, so a screenshot in a document is the website's own screen
- The words each trade uses, as an overlay that may change vocabulary and never shape
- Every technical word explained in plain language on first use

How you know it is done. node brand/site/build.js — the edition overlay is applied and the structural shape compared before and after. The build fails if a trade's words changed a module number, an app name or an app count.

Where this is written down: `brand/site/shots.js` · `brand/site/uishot.js` · `brand/site/plainwords.js`

---

4 · Development

Building it, module by module, in the order the modules are numbered. A module is not finished when its screens exist — screens can be demonstrated. It is finished when its rules hold and a test proves each one.

That is why the rulebook is the spine of this stage rather than an appendix to it. Every rule states what the system does AND what it refuses to do instead, because the refusal is the half a business relies on when nobody is watching.

What this stage owes

- Every rule for the module, satisfied and proven
- A test that fails before the fix and passes after it
- No count, rate, threshold or name belonging to a customer compiled into the code
- The build order respected — the spine before anything that stands on it

How you know it is done. npm test — every register gate and every engine check in one command. A rule marked enforced must name a file that exists and a test string findable inside it.

Where this is written down: `brand/site/guide.js` · `brand/site/rules.js` · `brand/site/modules.js`

---

5 · Infrastructure

What it runs on, and what it would take to run on something else. Each layer names a default, at least two alternatives, and the interface everything above it talks to — so the promise “you are not locked in” is a sentence with a table behind it rather than a digit with nothing behind it.

Free options come first, and a paid tool has to name both the free option it replaces and the trigger that justifies the spend. A tool register with no free column is a shopping list.

What this stage owes

- Every layer with its default, its alternatives and its interface
- The switching cost of each, stated rather than implied
- A free option named for every capability
- Any paid tool naming what it replaces and when it becomes worth it

How you know it is done. node brand/site/checkstack.js --summary and node brand/site/checktools.js --summary — 19 layers with their swaps, and every paid tool naming its free option and its trigger.

Where this is written down: `brand/site/stack.js` · `brand/site/tools.js` · `DEPLOYMENT.md`

---

6 · Security

The isolation is in the database, not in the application. Row-level security means one business physically cannot read another’s records even when the code above has a bug — and the check that matters is not that policies exist but that the connecting role is subject to them. A superuser bypasses every policy, force or no force, so a system whose application connects as its owner has policies that are decoration.

That is proven by loading the schema into a real PostgreSQL and asking for one company’s rows as three different roles. Two of the three see everything.

The promises that follow are absolute rather than configurable: the system never asks for a marketplace, bank or account password; personal and banking details are read into memory for a computation and never written into a file the code repository can see; keys are entered at runtime and never committed.

What this stage owes

- Isolation enforced by the database, with the application role neither superuser nor owner
- An audit row for every change, carrying what it was as well as what it became

- No credential, key or personal detail in any committed file
- A trade configuration that may never switch off a guarantee

How you know it is done. node core/tests/live.test.js — the schema RUN, not read: real PostgreSQL, real policies, and the app role proven to be neither superuser nor table owner.

Where this is written down: `core/schema.postgres.sql` · `brand/site/rules.js` · `DEPLOYMENT.md`

---

7 · Testing

A gate only checks what somebody thought to ask it, so the question here is not “do the tests pass” but “would they fail”. Every check in this repository is proven by planting the failure it is supposed to catch and confirming it fires. A plant that fires nothing means the check is decoration, and several were rewritten after exactly that.

The registers are checked as hard as the code. A rule may not claim a proof it does not have; a document may not use a technical word it never explains; a delivered PDF may not be older than the file it was rendered from; a count may not be typed where it could be derived.

What this stage owes

- Every check proven red before it is trusted green
- The negative control — a test that can fail, demonstrated failing
- Registers gated as strictly as code
- Counts derived at generation time, never typed

How you know it is done. npm test exits 0, and each gate has a documented plant that makes it exit non-zero.

Where this is written down: `brand/site/checkrules.js` · `brand/site/checkcoverage.js` · `core/tests/core.test.js`

---

8 · Deployment

From an empty machine to a running system: the database and its three roles, the schema applied as ordered migrations that are never edited once applied, the application service, the web server in front of it, certificates, and the automatic checks that run on every change.

Migrations are the part people get wrong twice. Once by editing an applied migration, which makes two installations diverge silently; once by having no way to roll one back.

What this stage owes

- Ordered migrations, never edited after they are applied
- Three database roles: owner, policy subject, and the application’s own

- The service, the web server in front of it, and certificates
- Automatic checks on every change, running the same suite a person runs

How you know it is done. DEPLOYMENT.md followed end to end on a clean machine, and the deployed system passing the same npm test the developer ran.

Where this is written down: `DEPLOYMENT.md` · `brand/site/guide.js`

---

9 · Monitoring

A system with no monitoring does not fail loudly. It fails on a Sunday, and the business finds out on Monday from a customer.

So what is watched is written down, each signal with the level that counts as trouble and the name of the person it wakes: disk left, error rate, queue depth, response time, failed sign-ins, and the age of the last successful backup. An alert carries the signal, the value, the level it passed and the first step to take — paging somebody with a number and no instruction turns every incident into a research project starting from zero at three in the morning.

And the backup rule, which is the one that costs the most when it is skipped: a backup is not a backup until it has been restored. Counting a job that exited zero as protection is how a business discovers on the worst day of its life that it has been writing an empty file nightly.

What this stage owes

- Each watched signal with its level and the person it wakes
- Alerts that name the first step, not just the number
- A restore drill on a stated interval, comparing row counts against the source
- An audit retention period read from configuration rather than assumed

How you know it is done. The restore drill runs on its interval and the restored row counts match the source. That drill is not yet written — see the gap below.

Where this is written down: `brand/site/stack.js` · `brand/site/rules.js`

> **What is not covered yet.** *This stage was empty until now. A search across the rulebook, the build guide, the*

architect document and the deployment runbook returned zero matches for “monitor”, “alert”, “launch” and “rollout”; the whole of it was one stack layer called Watching it and a backup section in the runbook. The rules are written and numbered now, and all of them are SPECIFIED rather than ENFORCED: nothing here is proven by a test yet. The highest-value next piece of work in this stage is the restore drill, because it is the one rule that can be made real with a script rather than a decision.

---

10 · Launch

Going live is a sequence, not a date. The old numbers and the new ones have to agree before anybody stops using the old way: totals, row counts and opening balances compared against the source, and every difference explained rather than waved through. A difference that was noticed and accepted becomes a figure somebody has to defend later with no record of where it came from.

Then both run at once over the same period and their outputs are compared, and the old way is retired only when they agree — a cutover on a date rather than on evidence makes the first real disagreement a production incident instead of a finding.

And the way back is written down and practised on a copy before it is needed, including what happens to records created after the release. A rollback plan that was written and never run is a plan in the same sense as an untested backup: believed until the one moment it has to work.

What this stage owes

- A reconciled migration — every difference against the old system explained
- A parallel run, with both outputs compared over the same period
- A cutover decided by evidence rather than by a date
- A rollback rehearsed on a copy, including records created after the release

How you know it is done. The parallel run agrees, the reconciliation has no unexplained difference, and the rollback has been performed once on a copy.

Where this is written down: `brand/site/rules.js`

> **What is not covered yet.** *Like monitoring, this stage did not exist in any register before now and the rules in it*

are all SPECIFIED. Nothing here has been done for a real business yet, and the sequence above is a design rather than a report. The tenant's own build guide carries an ordered setup path from signing up to running live, which is the nearest thing to a rehearsal that exists today.

Part two — the 22 modules, in full

In the order they are numbered, which is the order they are built. Not reordered by dependency or by size — the numbering is the build order already, and re-sorting a list somebody numbered is a second opinion nobody asked for.

Each module carries what it reads and what it writes, every app it contains with its purpose and its state, every rule it must satisfy in full, and its specified screens where they exist.

> **module** — *One area of work in the system — sales, purchase, staff, accounts. Each is a set of screens that belong together.* *Dukaan ke alag-alag counters. Ek counter bikri ka, ek kharidi ka, ek hisaab-kitaab ka.* **row** — *One single record — one customer, one order, one payment.* *Register mein ek line. Ek line matlab ek entry.* **table** — *One kind of information inside the database — all your customers in one, all your orders in another.* *Almari ka ek khaana. Ek khaane mein sirf customers, doosre mein sirf orders.* **audit trail** — *An automatic record of every change — what changed, who changed it, and when.* *Har entry ke saath naam aur time apne aap likha jaata hai. Baad mein koi bole "maine nahin kiya", toh register bata deta hai.*

#	Module	Apps	Rules	Proven
01	Platform *(spine)*	8	33	21
02	Design & Sampling	2	7	0
03	Inventory & Catalog	4	14	7
04	CRM	4	9	0
05	Sales	8	18	6
06	Planning & Requirements (MRP)	3	8	0
07	Purchase	3	12	0
08	Manufacturing	4	20	9
09	Quality & Compliance	2	7	0
10	Warehouse	3	8	0
11	Logistics	5	11	0
12	Accounting & GST	9	24	16
13	Treasury & Financial Planning	3	8	1
14	Settlement	3	13	0
15	E-commerce / OMS	11	19	10
16	HR & Payroll	5	22	8
17	Marketing	8	10	0
18	AI Content Engine	8	11	2
19	SEO, AEO & AIO	3	6	0
20	Projects & Collaboration	7	9	1
21	Dashboard & BI	5	9	6
22	AI Assistant, Agents & Automation	5	15	2

Module 01 · Platform

The spine the whole house runs on

Not a module you open — the layer underneath all 22. Who can see what, how Vastrangam is configured, and a record of everything that ever happened.

Reads from	Every module
Writes to	Every module
Apps	8
Rules	33, of which 21 are proven by a test that runs

The 8 apps in this module

Identity, Settings & Audit — SPECIFIED — designed, not built

Users, roles and permissions, company switching, tax and numbering setup — and an immutable record of everything that ever happened.

Industry Packs — SPECIFIED — designed, not built

What your trade calls things, the stages your work moves through, the extra fields your records need, the documents you issue and the reference data you start with — all of it loaded as a row of configuration, never as a separate version of the software. Pick the pack that matches your trade and the whole system changes its words: the same order screen reads as a job, a matter, a consignment or an appointment, with the same columns underneath. A pack may rename what exists, add fields to tables that exist, and switch discretionary rules off; it may never invent a concept, add a field to a table that does not exist, contain any executable code, or switch off the guarantees — company scoping, the audit trail, money never being a float — because a trade may change its vocabulary and may not opt out of the things the books are trusted for. Adding a trade nobody anticipated is therefore a file somebody writes, not a release somebody ships.

Ask & Print — **BROWSER APP** — opens and self-tests, no shared database behind it

At an exhibition in Hyderabad, send one line from your phone: “ledger Kalamandir”, “print slips”. It comes back as a PDF, or it prints at the Surat office — with nothing plugged into your phone and nothing at the office open to the internet.

Communications — SPECIFIED — designed, not built

WhatsApp commands and broadcasts, email and SMS, and the handful of scheduled jobs that carry a nudge to the right person without anyone remembering to send it. Every capability here has more than one interchangeable provider — none of them is ever the source of a figure the business reports.

WhatsApp Command Console — SPECIFIED — designed, not built

The shop floor does not open a laptop. A karigar or a staff member sends a short message — in, out, today’s pieces, leave, an advance — and it becomes a real record: attendance with the time and place it was marked, a

production report against the design, a request in the approvals queue. The end-of-day prompt asks what is still open and how long it needs, so tomorrow starts from what is actually pending rather than from memory. Marking attendance checks the phone is at the unit within the radius set for it, with a grace window; a person outside it is not refused, they are flagged for the manager, because a system that locks someone out of being paid for being early at the wrong gate has failed at its job. Every override is recorded with who made it. The words a worker types are in the language they speak, and nothing here ever asks for a password, a bank detail or a document number.

Data Privacy & Consent — SPECIFIED — designed, not built

What a person's data may be used for, captured as consent at the point it is given and honoured everywhere downstream — including the right to have it corrected or removed. Retention (how long a record is kept) and deletion (a person's right to have their own data removed) are two different policies, tracked separately, because a rule that keeps records for the law and a request from a person to be forgotten do not resolve the same way.

Provider Router & Cost Guard — **ENGINE** — the arithmetic runs on the command line, no screen yet

The rule that no capability depends on one outside service, enforced at the moment it matters instead of merely promised. Every capability has an ordered fallback list ending on an option that needs nothing connected, so a courier API that stops answering at 9pm, or an AI key that hits its quota mid-catalogue, drops to the next option instead of stopping the work. A provider that keeps failing is tripped out of the list entirely and retried once after a cooldown rather than hammered; retries inside one provider wait twice as long each time. And every paid call is counted in paise against a ceiling you set — over the ceiling the paid provider is refused, not warned about, and the work completes on a free one. Because every capability is guaranteed a built-in or by-hand option, a spent budget can stop the spending without ever stopping the business.

Payment Data Scope — SPECIFIED — designed, not built

A written statement of exactly which systems ever see a card or bank credential, and which never do — because every card-capable screen in this system hands that moment to a payment provider's own secured field and never stores or even passes the number through application code. The statement is what an auditor or a partner asks for before they will connect to this system.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Roles and permissions · who may do what at Vastrangam

- **Figures across the top** — Users 46 · Roles defined 7 · Companies 3 · Money by message Never
- **Columns** — Role · Can see · Can change · Can approve
- **Rows** — 5 worked examples, the first reading: Owner · Everything · Everything · No limit
- **Controls** — Actions carrying a named user · Documents sent out need a one-time code

The 33 rules this module must satisfy

`R01.1` Every business record names the company it belongs to

- **When** any record is written — a sale, a movement, a voucher, an employee

- **Then** its company is stored on the row itself
- **Never** inferring the company from who happens to be logged in, which silently mis-files every record made by someone who works across two of them
- **Proven** by `core/tests/core.test.js › the schema loads and the three companies keep three different codes`

`R01.2` One company cannot read another company’s records

- **When** a query runs for a user scoped to one company
- **Then** rows belonging to any other company are not returned at all
- **Never** filtering in the screen while the data is already loaded — a filter can be removed, a scope cannot
- **Proven** by `core/tests/core.test.js › one company cannot read another company`

`R01.3` The audit trail has no off switch

- **When** anything touching money, stock, price, tax, pay or master data changes
- **Then** the change and its before-image are written in the same transaction as the change itself
- **Never** allowing a setting, a role or a migration to disable it — either both land or neither does
- **Proven** by `core/tests/core.test.js › an audited insert leaves a before/after trail`

`R01.4` An update records what it was, not only what it became

- **When** a row is changed
- **Then** the current row is read first so the before-image is what was really there
- **Never** trusting the caller’s idea of the old value, which makes the trail a record of intentions rather than of facts
- **Proven** by `core/tests/core.test.js › an update records what it was as well as what it became`

`R01.5` A table nobody thought to audit is refused

- **When** code writes to a table that is not on the audited list
- **Then** the write is refused and the table is named
- **Never** letting it through quietly, which is how a money column ends up outside the trail without anyone deciding that
- **Proven** by `core/tests/core.test.js › a table nobody thought to audit is refused, rather than slipping through`

`R01.6` Deletion is a reversal, never a removal

- **When** a user deletes anything
- **Then** the record is voided, marked, and still readable with the reason
- **Never** removing the row — eight years of trail cannot survive a DELETE
- **Proven** by `core/tests/core.test.js › voiding is the only removal, and it is reversible`

`R01.7` A module that is not in the canonical list cannot join the bus

- **When** code subscribes to a business event
- **Then** the module number is checked against modules.js and refused if unknown

- **Never** letting an unregistered listener attach, which is how a cascade gains a step nobody can find later
- **Proven** by `core/tests/core.test.js › a module not in modules.js cannot subscribe`

`R01.8` A cascade is all of it or none of it

- **When** one business event fans out to stock, ledger, customer and documents
- **Then** every step commits together, or the whole thing is rolled back
- **Never** leaving stock moved and the ledger unposted, which is the exact state no report can ever explain
- **Proven** by `core/tests/core.test.js › a sale moves stock and posts to the ledger, or does neither`

`R01.9` A handler that throws takes the transaction with it

- **When** any subscriber to an event fails
- **Then** the emitting transaction fails too
- **Never** swallowing the error so the originating action appears to have succeeded
- **Proven** by `core/tests/core.test.js › a handler that throws takes the whole transaction with it`

`R01.10` No capability depends on a single outside service

- **When** any capability is used — books, courier, payments, AI, storage, GST
- **Then** an ordered list of interchangeable providers is tried, ending on one that needs nothing connected
- **Never** having one provider whose outage stops the work, however good that provider is
- **Proven** by `brand/suite/router.js › no spend ceiling can exhaust any cascade (a free option is always in it)`

`R01.11` A failing provider is taken out of the list, not hammered

- **When** a provider fails repeatedly
- **Then** it is tripped open, skipped entirely, and retried once after a cooldown
- **Never** retrying into a dead service on every call while a working alternative sits further down the list
- **Proven** by `brand/suite/router.js › three consecutive failures trip the breaker open`

`R01.12` A spend ceiling refuses, it does not warn

- **When** a paid call would take spending past the ceiling set for it
- **Then** that provider is refused and the work completes on a free one
- **Never** letting it through with a warning nobody reads, and never refusing only the first provider while the same spend reroutes to the next
- **Proven** by `brand/suite/router.js › a ceiling below the price refuses every paid option, not just the first`

`R01.13` The system never asks for a marketplace, bank or account password

- **When** any integration is connected, by any module, including a chatbot or an agent
- **Then** a scoped, revocable key is requested instead, cancellable from the provider’s side without changing the login

- **Never** accepting, storing, echoing or transmitting an account password — there is no screen, no import and no support flow that takes one
- **Not proven yet** — specified, no test behind it

`R01.14` Card and bank credentials never reach application code

- **When** a payment needs a card or bank detail
- **Then** the provider's own secured field takes it directly
- **Never** passing it through this system, even in transit, even unlogged — what is never received cannot be leaked
- **Not proven yet** — specified, no test behind it

`R01.15` Consent and retention are two different clocks

- **When** a person's data is held
- **Then** why it may be used and how long it is kept are tracked separately, and an erasure request is resolved against both
- **Never** treating a legal retention period as consent to keep using the data for anything else
- **Not proven yet** — specified, no test behind it

`R01.16` A scoped key is revocable without touching the login

- **When** an outside service is connected
- **Then** a key limited to what that capability needs is stored, and the connection records which capability it serves
- **Never** storing a credential that can do more than the capability requires, because the day it leaks is the day that difference matters
- **Not proven yet** — specified, no test behind it

`R01.17` A webhook is verified, idempotent and never silently dropped

- **When** a payment, courier, storefront or messaging provider calls in
- **Then** the signature is checked, the external id makes a repeat delivery a no-op, and a failure is logged with its payload for retry
- **Never** trusting an unsigned call, and never processing the same external id twice — a duplicated payout or a duplicated order is indistinguishable from a real one afterwards
- **Not proven yet** — specified, no test behind it

`R01.18` A trade is added as data, never as a version of the software

- **When** a business in a trade the system has never seen signs up
- **Then** its vocabulary, stages, extra fields, documents, rule switches and starting reference data arrive as one configuration file, and every screen reads back in that trade's words
- **Never** a branch, a fork or a bespoke build per industry — that is a consultancy with software attached, and it is the thing that stops a product from being one
- **Proven** by `core/tests/packs.test.js › GATE · it loads from a JSON string with no code change`

`R01.19` A pack is data and can never be code

- **When** a pack is loaded from any source
- **Then** every value in it is inspected, at any depth, and a function anywhere inside it refuses the whole pack
- **Never** letting configuration carry behaviour — the moment a pack can run code, adding a trade is a code change again and the guarantee in R01.18 is worthless
- **Proven** by `core/tests/packs.test.js › a pack containing a function`

`R01.20` A pack may rename a concept, never invent one

- **When** a pack declares its vocabulary
- **Then** each entry is matched against the fixed list of concepts the engine has, and an unknown one refuses the pack
- **Never** accepting an unrecognised word as a new concept, which turns "vocabulary" into a place to put anything and leaves the screens with a name for something that does not exist
- **Proven** by `core/tests/packs.test.js › renaming a concept the engine does not have`

`R01.21` A pack extends tables that exist, and nothing else

- **When** a pack adds fields
- **Then** the table is checked against the real schema and the field type against the types the engine can store
- **Never** creating a table on a customer’s behalf from a configuration file, which puts the shape of the database outside the reach of the schema test that guards it
- **Proven** by `core/tests/packs.test.js › adding a field to a table that does not exist`

`R01.22` Money in a pack is money everywhere else

- **When** a pack adds a field whose name reads as an amount, a price, a cost, a total, a fee or a rate
- **Then** it must be declared in paise, and a plain number refuses the pack
- **Never** letting a trade introduce a floating-point rupee through the side door after the whole schema was built to keep them out
- **Proven** by `core/tests/packs.test.js › money declared as a plain number`

`R01.23` No pack can switch off a guarantee

- **When** a pack sets a rule off
- **Then** the rule id is checked against the rulebook, and against the list of rules no pack may touch — company scoping, the audit trail, the posting rules, group elimination and roster privacy
- **Never** a trade opting out of the things that make the books trustworthy; it may call an invoice whatever it likes and may not decide its trail is optional
- **Proven** by `core/tests/packs.test.js › switching OFF the audit trail`

`R01.24` A rule a pack never mentions is on

- **When** a rule is looked up for a trade
- **Then** the rulebook is the default and the pack is read as an exception list — silence means the rule applies

- **Never** treating the pack as a permission list, which would mean every rule added after a pack was written silently applies to nobody who is using it
- **Proven** by `core/tests/packs.test.js › a rule the pack never mentions is ON — a pack is an exception list, not a permission list`

`R01.25` An invalid pack is refused whole, never half-loaded

- **When** a pack fails any check
- **Then** every problem in it is reported at once and none of it is applied
- **Never** partially loading a trade, which leaves a system whose vocabulary and rules disagree with each other and no way to tell which half is live
- **Proven** by `core/tests/packs.test.js › a refused pack is refused whole — nothing is half-applied`

`R01.26` A backup is not a backup until it has been restored

- **When** a backup is taken on its schedule
- **Then** it is restored into a scratch database on a stated interval and the row counts of the restored copy are compared against the source
- **Never** counting a backup as protection because the job exited zero, which is how a business discovers on the worst day of its life that it has been writing an empty file nightly
- **Not proven yet** — specified, no test behind it

`R01.27` What is watched is written down, with the number that means trouble

- **When** the system runs in production
- **Then** each watched signal names the level that counts as trouble and the person it wakes — disk left, error rate, queue depth, response time, failed sign-ins, and the age of the last successful backup
- **Never** a dashboard nobody is on the hook for, which is a screen that gets looked at after somebody has already noticed by other means
- **Not proven yet** — specified, no test behind it

`R01.28` An alert names what to do, not just what is wrong

- **When** a watched signal passes the level that counts as trouble and somebody is woken
- **Then** the alert carries the signal, the value, the level it passed, and the first step to take
- **Never** paging a person with a metric and no instruction, which turns every incident into a research project starting from zero at three in the morning
- **Not proven yet** — specified, no test behind it

`R01.29` A change records what it was, as well as what it became

- **When** a business record is created or changed
- **Then** the audit row carries the previous value, the new value, who made the change and when
- **Never** logging only the new value, which tells you the number is wrong today and nothing about what it was before somebody changed it
- **Proven** by `core/tests/core.test.js › an update records what it was as well as what it became`

`R01.33` The audit trail is kept for a period somebody chose

- **When** audit rows age past the retention period
- **Then** the period is read from configuration, and anything removed is removed on that stated rule
- **Never** a retention that exists only in whoever set up the log rotation, which is how the one month somebody needs is the one month that was rotated away
- **Not proven yet** — specified, no test behind it

`R01.30` Nothing goes live until the old numbers and the new ones agree

- **When** data is migrated from whatever the business ran before
- **Then** the migration reconciles: totals, row counts and opening balances are compared against the source and every difference is explained before cutover
- **Never** going live on a migration whose differences were noticed and waved through, because every one of them becomes a figure somebody has to defend later with no record of where it came from
- **Not proven yet** — specified, no test behind it

`R01.31` The old way keeps running until the new one has agreed with it

- **When** a module is ready to go live
- **Then** both run over the same period and their outputs are compared, and the old one is retired only after they agree
- **Never** a cutover on a date rather than on evidence, which makes the first real disagreement a production incident instead of a finding
- **Not proven yet** — specified, no test behind it

`R01.32` A rollback is rehearsed before it is needed

- **When** a release is prepared
- **Then** the way back is written down and practised on a copy, including what happens to records created after the release
- **Never** a rollback plan that was written and never run, which is a plan in the same sense as an untested backup — believed until the one moment it has to work
- **Not proven yet** — specified, no test behind it

Module 02 · Design & Sampling

A style exists on paper before it exists as stock

A product moves from a first idea to something the business can actually make and sell — specification, sample rounds, costed trials and sign-off — before it is ever entered as a catalog record. Building this first means Inventory & Catalog never has to invent a style it has no real specification for.

Reads from	CRM
Writes to	Inventory & Catalog, Manufacturing
Apps	2
Rules	7, of which 0 are proven by a test that runs

The 2 apps in this module

PLM & Development — SPECIFIED — designed, not built

Concept to a design that can actually be made: fabric and trim specification, sample rounds with the mill, costed trials against a target price, and sign-off — every version kept, so last season's costing is still there.

Design / IP Register — SPECIFIED — designed, not built

What protects a design once it exists — trademark or copyright status, the date it was first shown, and a flag the moment a near-identical listing turns up elsewhere. Every version already kept by PLM & Development gets a filed status here instead of just a date stamp, because a design with no ownership record on file is a design nobody can defend.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Design development · V-1180 · sample round 3

- **Figures across the top** — Designs in development 24 · At approval 6 · Rounds this design 3 · Cost vs target +4%
- **Columns** — Design · Round · What changed · Stage
- **Rows** — 4 worked examples, the first reading: V-1180 Anarkali · r3 · Kali width 22" → 24", flare corrected · At approval
- **Controls** — Sample rounds kept in full · Designs listed without sign-off

The 7 rules this module must satisfy

`R02.1` A style becomes a SKU only after sign-off

- **When** someone tries to create a catalogue record for a design
- **Then** the design must already have passed sample sign-off
- **Never** letting a SKU exist for something with no agreed specification, which puts an unmakeable item on sale
- **Not proven yet** — specified, no test behind it

`R02.2` Every version of a specification is kept

- **When** a sample round changes a measurement, a fabric or a trim
- **Then** a new version is written and the old one stays readable

- **Never** editing the specification in place — a worker paid against last month's spec must still be able to show what it said
- **Not proven yet** — specified, no test behind it

`R02.3` A costed trial carries the date its rates came from

- **When** a sample is costed
- **Then** the rate and the date it was in force are both stored on the trial
- **Never** recosting an old trial with today's rates and presenting the result as what it cost then
- **Not proven yet** — specified, no test behind it

`R02.4` A design with no ownership record is flagged, not blocked

- **When** a design reaches sign-off with no trademark or copyright status on file
- **Then** it proceeds and is listed as unprotected
- **Never** silently treating it as protected, which is only discovered when a near-identical listing appears and there is nothing to act on
- **Not proven yet** — specified, no test behind it

`R02.5` The first-shown date is recorded when it happens

- **When** a design is first shown publicly — an exhibition, a listing, a lookbook
- **Then** that date is stamped and never editable afterwards
- **Never** backdating it later, which is precisely the field a dispute turns on
- **Not proven yet** — specified, no test behind it

`R02.6` A rejected sample keeps its reason

- **When** a sample round is rejected
- **Then** the reason is recorded against the version
- **Never** closing it with a status alone, which loses the only information that stops the same mistake in the next round
- **Not proven yet** — specified, no test behind it

`R02.7` A specification cannot be deleted while stock exists against it

- **When** someone removes a design that has ever been made
- **Then** it is archived and stays linked to every piece produced from it
- **Never** orphaning finished stock from the specification it was made to
- **Not proven yet** — specified, no test behind it

Module 03 · Inventory & Catalog

One stock number everyone trusts

The most important number in the house: one quantity per design and size, per godown, per stage — greige, dyed, in stitching, finished, listed. Read and written by every other module. And one product record every marketplace lists from.

Reads from	Design & Sampling, Every module
Writes to	Every module
Apps	4
Rules	14, of which 7 are proven by a test that runs

The 4 apps in this module

Stock — **RUNNING** — on the real database, with a test that drives it

Live quantity by design, size and location, fabric in metres and pieces in numbers, with reorder alerts, lot tracking, set kits and dead-stock ageing.

> *Proven by* `medhava/test/inventory.test.js`.

Catalog / PIM — SPECIFIED — designed, not built

One record per design — fabric, work, length, colour, size chart, images, HSN, MRP and what each panel actually sells it at — scored for Myntra and Amazon readiness before it lists. It also carries the two things everything downstream needs: the code each panel knows the design by, mapped to yours, and the packed size and weight that decide the courier rate and settle every weight dispute. Every listing's state on every panel is here too — live, waiting for approval, blocked, archived — with the quality score that decides whether anyone sees it.

Kit & Combo SKU — SPECIFIED — designed, not built

A sellable SKU made of component SKUs — a three-piece set sold as one listing. Selling the kit decrements each component at order time, so stock is right for every piece in the set, not just the set itself.

Master-Data Hygiene — SPECIFIED — designed, not built

Duplicate detection and merge across customers, vendors and designs, and a dead-stock register for what has not moved in months. Protects every downstream report from the same record existing twice under two names.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Design record · one design, every panel's name for it

- **Figures across the top** — Designs live 2,418 · Mapped on all panels 2,301 · Missing size or weight 37 · Below reorder 44
- **Columns** — Your code · Panel · Their code · Packed size · weight · Sells at

- **Rows** — 4 worked examples, the first reading: VS_MuskanPurple_S · Myntra · VARMKASS136375444 · 30×25×3 cm · 0.45 kg · ₹1,947
- **Controls** — Codes mapped on every live panel · Size and weight on file

The 14 rules this module must satisfy

`R03.1` Stock is one number per SKU, per location, per stage

- **When** any module asks how much there is
- **Then** it reads the one quantity, with the channel recorded on the movement rather than on the stock
- **Never** keeping a separate stock figure per channel — the last piece sold on one marketplace has to vanish from the other ten at the same instant, which per-channel inventory cannot do
- **Proven** by `core/tests/core.test.js › stock is one number per SKU, with the channel recorded on the movement`

`R03.2` Negative stock is a fault, not a state

- **When** an issue would take a quantity below zero
- **Then** the issue is refused
- **Never** recording a negative balance and leaving someone to explain it at month-end
- **Proven** by `core/tests/core.test.js › issuing more than exists is refused — negative stock is a fault, not a state`

`R03.3` Selling a kit decrements every component

- **When** a kit or combo SKU is sold
- **Then** each component SKU is decremented at order time
- **Never** decrementing only the kit, which leaves the components sellable twice
- **Proven** by `core/tests/core.test.js › selling a kit decrements every component`

`R03.4` A kit with no components is refused

- **When** an item is marked a kit but lists nothing
- **Then** the record is refused and named
- **Never** accepting it and silently decrementing nothing on every sale
- **Proven** by `core/tests/core.test.js › a kit that lists no components is refused, not silently sold as nothing`

`R03.5` Stock value ties to the item cost, always

- **When** stock is valued
- **Then** the value is computed from the quantity and the item cost
- **Never** storing a valuation that can drift from the quantity it is supposed to describe
- **Proven** by `core/tests/core.test.js › stock value ties to the item cost`

`R03.6` Every movement has a source, a destination, or both

- **When** a stock movement is recorded
- **Then** at least one end is named
- **Never** accepting a movement from nowhere to nowhere, which is how quantity appears without a cause
- **Proven** by `core/tests/core.test.js › a movement with neither a source nor a destination is refused`

`R03.7` A quantity is a whole number above zero

- **When** a movement is written
- **Then** a non-integer or non-positive quantity is refused
- **Never** accepting a negative movement as a shorthand for a reversal — a reversal is its own movement with its own reason
- **Proven** by `core/tests/core.test.js › a quantity must be a whole number above zero`

`R03.8` Goods in someone else's warehouse are still yours

- **When** stock sits in a channel's own warehouse under consignment or sale-or-return
- **Then** that warehouse is a location like any other and the stock is counted, valued and aged there
- **Never** letting it drop off the books until it sells, which understates both stock and exposure
- **Not proven yet** — specified, no test behind it

`R03.9` Fabric in metres and pieces in numbers share one item master

- **When** an item is defined
- **Then** its unit of measure is a property of the item
- **Never** building a second item master for a second unit, which splits the one stock number this module exists to protect
- **Not proven yet** — specified, no test behind it

`R03.10` A listing needs the packed size and weight before it can go out

- **When** a product is pushed to a channel
- **Then** packed dimensions and weight must be present
- **Never** listing without them, because that is the field every courier weight dispute is settled on
- **Not proven yet** — specified, no test behind it

`R03.11` The channel's own code for a product is mapped, not assumed

- **When** a product exists on a marketplace
- **Then** that channel's identifier is stored against ours
- **Never** matching on name or on a code we invented, which mis-posts every settlement line for that product
- **Not proven yet** — specified, no test behind it

`R03.12` A duplicate master record is merged, never left as two

- **When** the same customer, vendor or design is detected twice
- **Then** they are merged and both old identifiers keep resolving

- **Never** leaving two live records, which splits every total that record appears in
- **Not proven yet** — specified, no test behind it

`R03.13` A price is per channel and dated

- **When** a channel price is set
- **Then** it is stored against that channel with the date it takes effect
- **Never** holding one price and reading it as if it applied everywhere and always
- **Not proven yet** — specified, no test behind it

`R03.14` Dead stock is named as dead stock

- **When** an item has not moved for the period set for it
- **Then** it appears on the dead-stock register with its age and carrying value
- **Never** leaving it inside the general stock figure where it reads as healthy inventory
- **Not proven yet** — specified, no test behind it

Module 04 · CRM

Know every boutique, chain and customer completely

One record per party — a Kalamandir or a Rajmandir, a Surat walk-in or a Myntra buyer — carrying every enquiry, order, return, agreement and conversation, whichever channel it arrived on.

Reads from	Every module
Writes to	Sales, E-commerce / OMS, Marketing
Apps	4
Rules	9, of which 0 are proven by a test that runs

The 4 apps in this module

CRM & Customer 360 — **BROWSER APP** — opens and self-tests, no shared database behind it

Enquiry to confirmed order, then the full lifetime: what they bought, what came back, what they are worth and which new range to show them first.

Documents & eSign — **BROWSER APP** — opens and self-tests, no shared database behind it

Mill agreements, job-work contracts, signed delivery challans, export documents and boutique credit terms filed against the party or order they belong to — found by that record, not by hunting through a folder.

Helpdesk & Live Chat — **BROWSER APP** — opens and self-tests, no shared database behind it

A boutique asking where its parcel is, or a customer asking about a size — the question becomes a ticket tied to the order, with the whole history already open.

Forms & Feedback (NPS) — SPECIFIED — designed, not built

A short form after delivery, and the score it produces attached to the design or item it is actually about — not just the buyer — so a complaint-prone item surfaces as a pattern instead of a scatter of individual gripes.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Party 360 · Kalamandir Chain

- **Figures across the top** — Lifetime value ₹94.2 L · Open tickets 3 · Outstanding ₹4.1 L · On-time paid 88%
- **Columns** — Record · What it is · When · State
- **Rows** — 4 worked examples, the first reading: VS-SO-2291 · Order · 240 pcs, 6 designs · 12 Jul · Delivered
- **Controls** — First reply under 2 h · Resolved same day

The 9 rules this module must satisfy

`R04.1` One customer, one record, whichever channel they arrived by

- **When** the same person orders on a marketplace and later at the counter
- **Then** both land on one record with the channel noted on each order
- **Never** creating a second customer per channel, which makes lifetime value meaningless
- **Not proven yet** — specified, no test behind it

`R04.2` A document is filed against the record it belongs to

- **When** any agreement, receipt, certificate or scan is stored
- **Then** it is attached to the order, party, case or employee it concerns
- **Never** filing it in a folder that has to be remembered rather than found
- **Not proven yet** — specified, no test behind it

`R04.3` A signed copy files itself back

- **When** a document sent for signature is signed
- **Then** the signed version returns to the same record automatically
- **Never** leaving the signed copy in an inbox while the record still shows it as pending
- **Not proven yet** — specified, no test behind it

`R04.4` A ticket carries the order it is about

- **When** a question arrives by chat, email or phone
- **Then** it is tied to the order or account it concerns, with the history already on screen
- **Never** opening a ticket with no link, which makes the first reply a request to explain again

- **Not proven yet** — specified, no test behind it

`R04.5` Feedback attaches to the item, not only the buyer

- **When** a rating or complaint arrives after delivery
- **Then** it is attached to the design or item it is actually about
- **Never** holding it only against the customer, which hides a complaint-prone item as a scatter of unrelated gripes
- **Not proven yet** — specified, no test behind it

`R04.6` A customer’s consent travels with their data

- **When** a customer record is used for marketing or profiling
- **Then** the consent captured at the point it was given is checked first
- **Never** assuming that having the data implies permission to use it for anything
- **Not proven yet** — specified, no test behind it

`R04.7` A merged customer keeps both histories

- **When** two customer records are merged
- **Then** every order, ticket and document from both survives on the surviving record
- **Never** discarding the shorter history to make the merge simple
- **Not proven yet** — specified, no test behind it

`R04.8` Credit state is read at the moment of the order

- **When** a B2B order is placed
- **Then** the customer’s outstanding and limit are evaluated then
- **Never** using a figure cached from the last sync, which is how a party goes past its limit between refreshes
- **Not proven yet** — specified, no test behind it

`R04.9` A closed ticket keeps what resolved it

- **When** a ticket is closed
- **Then** the resolution is recorded on it
- **Never** closing with a status alone, which loses the answer the next identical question needs
- **Not proven yet** — specified, no test behind it

Module 05 · Sales

Counter, wholesale, website and export — one order book

The Surat counter, the boutique wholesale book, the website and the export shipment all write to the same order and draw on the same stock number. And the parcel is followed to the door, because a sale is not done until the COD money is in.

Reads from	Inventory & Catalog, CRM, Warehouse, Logistics
Writes to	Inventory & Catalog, Accounting & GST, Warehouse, Logistics
Apps	8
Rules	18, of which 6 are proven by a test that runs

The 8 apps in this module

D2C Sales — **RUNNING** — on the real database, with a test that drives it

Orders from your own storefront, cart to dispatch, with loyalty and partial COD on a ₹4,400 anarkali.

> *Proven by* `medhava/test/sales.test.js` .

B2B & Credit — **BROWSER APP** — opens and self-tests, no shared database behind it

Boutique and chain orders on credit limits and tier pricing, with outstanding aged against each party's own agreed terms.

Export — **BROWSER APP** — opens and self-tests, no shared database behind it

Commercial invoice, packing list, LUT bond and IGST-refund tracking for the Gulf and UK buyers.

POS — **BROWSER APP** — opens and self-tests, no shared database behind it

Counter billing at Udhna that draws on the same stock as the website — no second stock register.

Quotes & Proforma — **BROWSER APP** — opens and self-tests, no shared database behind it

Send a quote, convert it to a confirmed order in one click.

Couriers & AWB — SPECIFIED — designed, not built

Book the parcel on the order, compare couriers for that pin code, print the label with the design code on it, and follow the AWB to the door.

Subscriptions — SPECIFIED — designed, not built

A schedule that raises its own invoice on its cycle and follows up on its own when a payment fails — for anything sold as a standing order rather than a one-off.

Customisation & Made-to-Measure — SPECIFIED — designed, not built

The order that does not exist in the catalogue: a buyer sends reference pictures and their own measurements, a price is agreed over several messages, and the piece is made for them. All of it is one record — the references, the measurement set, every quote in the negotiation and what was finally agreed, the advance taken to start work and the balance taken before dispatch. When the order is accepted it opens a production order like any other, so a bespoke piece is costed, stitched, checked and posted exactly as a catalogue piece is. Two legs of money on one order is the part most systems get wrong: the advance is earned when the work starts, the balance is owed until

the piece ships, and the ledger shows both separately rather than one payment appearing when the whole thing is over.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Order book · counter, boutique, website, export

- **Figures across the top** — Orders today 312 · To dispatch 118 · Credit held 7 · Avg order ₹8,940
- **Columns** — Order · Party · Channel · Value · State
- **Rows** — 4 worked examples, the first reading: VS-SO-2489 · Rajmandir Wholesale · B2B · 45 days · ₹2,84,000 · Credit hold
- **Controls** — Dispatched within cut-off · COD collected

The 18 rules this module must satisfy

`R05.1` Every sale carries its company and its channel

- **When** an order is created on any channel
- **Then** both are written on the order
- **Never** leaving either to be inferred later from the document number or the warehouse
- **Proven** by `core/tests/core.test.js › every one of the hundred cells posted its own figure, channel by channel`

`R05.2` A sale posts stock and ledger together

- **When** a sale is confirmed
- **Then** stock is deducted, the invoice is raised, and the ledger is posted in one transaction
- **Never** invoicing without moving stock, or moving stock without posting
- **Proven** by `core/tests/core.test.js › a sale moves stock and posts to the ledger, or does neither`

`R05.3` If the ledger refuses, the stock never moved

- **When** the posting half of a sale fails
- **Then** the stock movement is rolled back with it
- **Never** leaving the goods gone and the books untouched
- **Proven** by `core/tests/core.test.js › and if the ledger refuses, the stock never moved`

`R05.4` A quote becomes an order without being retyped

- **When** a quotation is accepted
- **Then** the order is created from it, carrying the same lines and prices
- **Never** re-entering the lines, which is where the price on the quote and the price on the invoice start to differ
- **Not proven yet** — specified, no test behind it

`R05.5` A price below the floor needs an approval, not a note

- **When** a line is priced under the floor set for it
- **Then** the order waits in the approvals queue with the rule that stopped it named
- **Never** letting it through with a comment box, which is a discount policy nobody can enforce
- **Not proven yet** — specified, no test behind it

`R05.6` An export invoice knows it is an export

- **When** an order ships outside the country
- **Then** the LUT or IGST treatment, currency and shipping terms are set on the order itself
- **Never** treating it as a domestic invoice and correcting the tax afterwards
- **Not proven yet** — specified, no test behind it

`R05.7` A counter sale is the same order record

- **When** someone buys at the counter
- **Then** the same order table records it, with the counter as the channel
- **Never** running the till on a separate book that has to be merged later
- **Not proven yet** — specified, no test behind it

`R05.8` A credit sale reserves the credit at the moment it is taken

- **When** a B2B order is accepted on credit
- **Then** the exposure is committed against the party immediately
- **Never** counting it only when the invoice is raised, which lets several orders each fit inside the same limit
- **Not proven yet** — specified, no test behind it

`R05.9` A dispatch cannot exceed what was ordered

- **When** a shipment is prepared
- **Then** quantities are checked against the order line
- **Never** shipping over, which becomes an invoice the customer never agreed to
- **Not proven yet** — specified, no test behind it

`R05.10` A cancelled order releases what it held

- **When** an order is cancelled
- **Then** reserved stock and committed credit are both released
- **Never** leaving stock reserved against a dead order, which shows the business as out of goods it actually has
- **Not proven yet** — specified, no test behind it

`R05.11` An AWB belongs to the shipment, not the courier integration

- **When** a tracking number is recorded, typed in or fetched
- **Then** it is stored on the shipment

- **Never** making the number reachable only through whichever courier API produced it, which loses it the day that courier is dropped
- **Not proven yet** — specified, no test behind it

`R05.12` A subscription renewal is a new order

- **When** a subscription renews
- **Then** a fresh order is created with its own stock, invoice and posting
- **Never** extending the original order, which makes the revenue of two periods indistinguishable
- **Not proven yet** — specified, no test behind it

`R05.13` A sale to a sister company is marked as one

- **When** the counterparty is another company in the group
- **Then** the counterparty company is recorded on the entry
- **Never** posting it as an ordinary outside sale, which inflates the group turnover by trade it never did
- **Proven** by `core/tests/core.test.js › an entry cannot be its own counterparty`

`R05.14` A quote or proforma number carries its type and financial year

- **When** a quotation or proforma is raised
- **Then** it is numbered Q-{FY}-#### or PI-{FY}-####, sequential within that company and year
- **Never** sharing one sequence between quotations and proformas, which makes a proforma indistinguishable from a quote in the register
- **Not proven yet** — specified, no test behind it

`R05.15` A quote line with no description, no quantity or a negative rate is not a line

- **When** a quotation is totalled
- **Then** only lines with a description, a quantity above zero and a rate of zero or more are counted
- **Never** letting a half-filled row contribute a number to the total
- **Proven** by `medhava/test/inventory.test.js › I2 R03.7 · a fractional, zero or negative quantity is refused`

`R05.16` An export line carries no GST

- **When** a quotation or invoice is marked export under LUT
- **Then** the GST percentage is zero and the document says why
- **Never** applying the domestic rate and correcting it after the buyer queries the total
- **Proven** by `medhava/test/sales.test.js › S6 R05.16 · an export under LUT carries no GST, and says so on the invoice`

`R05.17` A made-to-measure order has two money legs, and both are visible

- **When** a customisation order is accepted
- **Then** the advance and the balance are recorded as separate amounts with their own dates, and the balance stays owed until dispatch

- **Never** showing one payment at the end, which hides money already taken and work already owed
- **Not proven yet** — specified, no test behind it

`R05.18` A customisation quote keeps every round of the negotiation

- **When** a price is revised during a bespoke enquiry
- **Then** each quoted figure is kept in order with what changed
- **Never** overwriting the earlier figure, which is the one the customer remembers agreeing to
- **Not proven yet** — specified, no test behind it

Module 06 · Planning & Requirements (MRP)

Turn what is selling into what to buy and make

Confirmed orders and demand history have to become a plan before Purchase can buy anything or Manufacturing can start anything — otherwise buying and making are both just guessing. This module sits between the two: it reads what is actually selling and what is already committed, and turns that into requirement, not the other way around.

Reads from	Sales, E-commerce / OMS, Inventory & Catalog
Writes to	Purchase, Manufacturing
Apps	3
Rules	8, of which 0 are proven by a test that runs

The 3 apps in this module

Demand Forecast & Signal — SPECIFIED — designed, not built

What sold, by SKU and by period, turned into a short-term forecast — the number every requisition and every production order downstream is measured against instead of a guess.

Requirement Explosion (MRP run) — SPECIFIED — designed, not built

Confirmed demand exploded through the bill of materials into what raw material to buy and what to produce, and by when — so a purchase requisition or a production order always traces back to real demand, never to a feeling that stock is getting low.

Open-to-Buy / Budget Ceiling — SPECIFIED — designed, not built

A spending ceiling set per period regardless of what the demand signal suggests, so a real signal cannot on its own commit more money than the business has decided to risk this season. Every requisition the MRP run drafts is checked against what is left of the ceiling before it goes to Purchase.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Requirement run · what to buy for the festive book

- **Figures across the top** — Shortfalls 31 · Already on order 12 · Buy this week ₹24.6 L · Budget left ₹8.4 L
- **Columns** — Material · Needed · On order · Order now
- **Rows** — 4 worked examples, the first reading: Chinon 44" · 4,200 m · 1,600 m · 2,600 m
- **Controls** — Shortfalls tracing to a real order or forecast · Material ordered twice

The 8 rules this module must satisfy

`R06.1` A forecast is labelled a forecast wherever it appears

- **When** a projected figure is shown beside actuals
- **Then** it is visually and structurally distinct
- **Never** letting a forecast total sit in the same column as a real one, which is how a plan becomes a reported result
- **Not proven yet** — specified, no test behind it

`R06.2` A requirement run reads live stock, not a snapshot

- **When** the MRP run explodes requirements
- **Then** it reads the current quantity at the moment it runs
- **Never** planning against a nightly copy, which orders material the business already has
- **Not proven yet** — specified, no test behind it

`R06.3` A requirement names what caused it

- **When** the run produces a shortfall
- **Then** the order, forecast or reorder level that generated it is recorded on the line
- **Never** producing a bare quantity nobody can trace back to a demand
- **Not proven yet** — specified, no test behind it

`R06.4` Stock already on order counts against the shortfall

- **When** the run computes what to buy
- **Then** open purchase orders are netted off first
- **Never** ignoring them and ordering the same material twice
- **Not proven yet** — specified, no test behind it

`R06.5` A budget ceiling refuses, it does not warn

- **When** a proposed purchase would exceed the open-to-buy ceiling
- **Then** it is held for approval with the ceiling named
- **Never** raising it with a warning, which makes the ceiling advisory and therefore not a ceiling

- **Not proven yet** — specified, no test behind it

`R06.6` A lead time is per vendor and per item

- **When** a run works out when to order
- **Then** it uses the lead time recorded for that vendor and that item
- **Never** applying one global lead time, which under-orders the slow lines and over-orders the fast ones
- **Not proven yet** — specified, no test behind it

`R06.7` A run is kept, not overwritten

- **When** the MRP run executes again
- **Then** the previous run stays readable with its inputs
- **Never** replacing it, which makes it impossible to see why last week's decision was taken
- **Not proven yet** — specified, no test behind it

`R06.8` A seasonal signal cannot silently become a permanent one

- **When** a festival or season inflates demand
- **Then** the period it applies to is stored with the signal
- **Never** folding a spike into the baseline, which keeps ordering for a festival all year
- **Not proven yet** — specified, no test behind it

Module 07 · Purchase

Nothing over-billed by a mill gets paid

The buy side end to end — mills, dyers, job workers and packing suppliers — with the control that stops you paying for metres you rejected.

Reads from	Inventory & Catalog, Planning & Requirements (MRP), Manufacturing
Writes to	Inventory & Catalog, Accounting & GST, Quality & Compliance
Apps	3
Rules	12, of which 0 are proven by a test that runs

The 3 apps in this module

Procurement — **RUNNING** — on the real database, with a test that drives it

Enquiry to purchase order to goods receipt, with a strict three-way match: you ordered 100 metres, 100 arrived, quality accepted 96, and the bill is only cleared for 96.

> *Proven by* `medhava/test/purchase.test.js` .

Vendor Management — **BROWSER APP** — opens and self-tests, no shared database behind it

Mill 360 — payables, ageing, a real risk score from accept rate and spend concentration, and sourcing that follows performance rather than habit.

Insurance Register — SPECIFIED — designed, not built

What cover exists over stock in transit, stock in the warehouse and product liability, matched against what is actually moving through Purchase and Warehouse right now — so real value in transit is never quietly running with no cover tracked anywhere.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Three-way match · nothing over-billed is paid

- **Figures across the top** — Open POs 62 · Bills held 5 · Payable to mills ₹41.2 L · Avg accept rate 94%
- **Columns** — Bill · Mill / job worker · Ordered · Accepted · Billed for
- **Rows** — 4 worked examples, the first reading: B-8841 · Jagdamba Textiles · 100 m · 96 m · 96 ✓
- **Controls** — Bills matched without a query · Spend with top mill

The 12 rules this module must satisfy

`R07.1` Nothing is paid without a three-way match

- **When** a vendor invoice is approved
- **Then** the purchase order, the goods received note and the invoice must agree
- **Never** paying on the invoice alone, which pays for goods that never arrived
- **Not proven yet** — specified, no test behind it

`R07.2` A short or damaged receipt is recorded as received short

- **When** the GRN quantity is below the PO quantity
- **Then** the difference is recorded with its reason and the payable follows the received quantity
- **Never** receiving the full quantity to make the match pass
- **Not proven yet** — specified, no test behind it

`R07.3` Input tax credit is claimed against a real document

- **When** ITC is taken on a purchase
- **Then** the vendor invoice and its tax detail are on file
- **Never** claiming credit from a payment record alone, which is the claim that fails reconciliation
- **Not proven yet** — specified, no test behind it

`R07.4` Landed cost reaches the item, not just the P&L

- **When** freight, duty or insurance is attached to a purchase
- **Then** it is apportioned into the cost of the items received
- **Never** expensing it separately, which understates the cost of every piece made from that material
- **Not proven yet** — specified, no test behind it

`R07.5` A vendor price is dated

- **When** a rate is agreed with a supplier
- **Then** it is stored with the date it takes effect
- **Never** overwriting the old rate, which makes last month’s purchase look mispriced
- **Not proven yet** — specified, no test behind it

`R07.6` A purchase order over its approval level waits

- **When** a PO exceeds the value a role may approve
- **Then** it goes to the approvals queue naming the rule and the level
- **Never** splitting it into smaller orders to fit under the limit — the split is detected and the parts are assessed together
- **Not proven yet** — specified, no test behind it

`R07.7` A vendor with no active record cannot be paid

- **When** a payment is raised
- **Then** the vendor must exist, be active, and have its bank detail verified
- **Never** paying to detail typed onto the payment itself, which is the single most common route for payment fraud
- **Not proven yet** — specified, no test behind it

`R07.8` A change to vendor bank detail is treated as high risk

- **When** a vendor’s bank account is changed
- **Then** the change is approved by a second person and the old detail is kept
- **Never** accepting a change from an email instruction alone
- **Not proven yet** — specified, no test behind it

`R07.9` A job-work despatch stays on the books

- **When** material is sent to a contractor
- **Then** it moves to a job-work location and remains this company’s stock
- **Never** writing it out on despatch, which loses material the business still owns
- **Not proven yet** — specified, no test behind it

`R07.10` An insurance policy is linked to what it covers

- **When** a policy is recorded
- **Then** the stock, premises or shipment it covers is named on it

- **Never** holding policies as documents with no link, which is discovered only at the moment of a claim
- **Not proven yet** — specified, no test behind it

`R07.11` The three-way match is arithmetic, not a judgement

- **When** a vendor invoice is checked
- **Then** the payable equals the received quantity × the purchase-order rate, and the purchase order, the goods receipt and the invoice must all agree on quantity and value
- **Never** passing an invoice whose value exceeds received quantity × agreed rate, and never letting an override happen without recording who made it and why
- **Not proven yet** — specified, no test behind it

`R07.12` A material is sourced down a ranked list, not from whoever answers

- **When** a material has to be bought
- **Then** the vendors ranked for that material are approached in their priority order
- **Never** defaulting to the last vendor used, which is how a price rise becomes permanent without anyone deciding
- **Not proven yet** — specified, no test behind it

Module 08 · Manufacturing

Know what a piece really costs to make

From the cut plan to the finished piece — what each karigar earned, what the dyer charged, what the zari cost, and what that design actually cost before you priced it.

Reads from	Purchase, Planning & Requirements (MRP), Design & Sampling
Writes to	Inventory & Catalog, HR & Payroll, Accounting & GST, Quality & Compliance
Apps	4
Rules	20, of which 9 are proven by a test that runs

The 4 apps in this module

Production Orders — SPECIFIED — designed, not built

Cutting, stitching, embroidery, washing, finishing and checking — your own stages, with work-in-progress visible at each and nothing lost at the dyer.

Piece-rate & Contractors — SPECIFIED — designed, not built

Karigars paid by the piece: pooled set completion, per-garment rates, alterations, rework and advances resolved into one payout.

BOM & Consumption — SPECIFIED — designed, not built

What each design consumes — metres of fabric, zari, lining, buttons, packing — costed at today's mill rates.

Maintenance — SPECIFIED — designed, not built

Machines and the building: what is due for service, when it was last done, what it cost, and what stopped while it was down.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Work in progress · cut to finish

- **Figures across the top** — Open production orders 48 · Pieces in WIP 1,260 · Rework 3.1% · Cost per piece ₹1,842
- **Columns** — Stage · Pieces in · Pieces out · Rejected · Ageing
- **Rows** — 4 worked examples, the first reading: Cutting · 420 · 406 · 14 · 1.2 d
- **Controls** — Accepted at first check · Orders finished on plan

The 20 rules this module must satisfy

`R08.1` Sets are pooled across every maker before the minimum is taken

- **When** completed sets are counted for a design
- **Then** every maker's pieces for that design are pooled first, and the set count is the minimum across the populated member columns of the pool
- **Never** counting sets per maker row and adding them up, which loses every set completed by two people between them
- **Proven** by `brand/suite/studio/verify_studio.js › pooling happens before the minimum, not per maker row`

`R08.2` A surplus piece is paid for, and is not a set

- **When** a maker produces more of one component than the set needs
- **Then** the extra is named individually and paid at its own piece rate
- **Never** adding it to the set count, and never leaving it unpaid because it did not complete a set — the person made it either way
- **Proven** by `brand/suite/studio/verify_studio.js › a surplus piece is named, is still paid for, and is never added to the sets`

`R08.3` A design counts on the components it actually has

- **When** a design is made of fewer component types than the usual set
- **Then** it is counted on the members it does have
- **Never** returning zero because an optional member is absent, which silently unpays a whole design

- **Proven** by `brand/suite/studio/verify_studio.js › a single-component design counts on what it has, not zero`

`R08.4` A missing rate posts zero and is flagged, never guessed

- **When** a design has no entry in the piece-rate master
- **Then** it costs zero and the design is named in the summary
- **Never** inferring a rate from a similar design — a guessed rate is a wrong payment to a real person
- **Proven** by `brand/suite/studio/verify_studio.js › a missing rate posts zero and is flagged, never guessed`

`R08.5` A two-row heading is read as two rows

- **When** the production grid uses a merged heading over component columns
- **Then** both header rows are read so repeated component names stay distinct columns
- **Never** reading only the first row, which collapses three same-named component columns into one and undercounts the work
- **Proven** by `brand/suite/studio/verify_studio.js › the two-row heading is read, so three same-named columns stay three components`

`R08.6` A worker written as a pair stays one unit

- **When** two names share one row as a working pair
- **Then** they are treated as a single paying unit
- **Never** splitting them into two workers, which halves each person’s recorded output and breaks the payout
- **Proven** by `brand/suite/studio/verify_studio.js › a maker written as a pair stays one unit`

`R08.7` Several years of grids pool into one set of figures

- **When** more than one production workbook is supplied
- **Then** their grids pool into a single costing
- **Never** reporting each file separately, which double-counts nothing but hides the sets completed across a year boundary
- **Proven** by `brand/suite/studio/verify_studio.js › several years of grids pool into one set of figures`

`R08.8` Cost per piece is independent of set completion

- **When** the cost of a design is worked out
- **Then** each raw piece is costed at its own rate
- **Never** costing by completed sets, which values an unfinished set at nothing while the labour has already been spent
- **Proven** by `brand/suite/studio/verify_studio.js › the grand total is the sum of the designs, and of the makers`

`R08.9` A production report moves stock and pay together

- **When** a piece-work production report is accepted
- **Then** finished stock comes in, the payout is raised in HR, wages post to the ledger, and the design cost updates — in one transaction
- **Never** taking the stock in and settling the pay in a separate pass, which is how the two disagree
- **Not proven yet** — specified, no test behind it

`R08.10` Material issued to production leaves raw stock at the moment it is issued

- **When** a production order consumes material
- **Then** raw stock is reduced and work in progress increases
- **Never** consuming at completion, which shows material as available while it is already cut
- **Not proven yet** — specified, no test behind it

`R08.11` A bill of materials is versioned with the design

- **When** a production order is created
- **Then** it captures the BOM version in force at that moment
- **Never** reading the current BOM when costing an old order, which recosts history
- **Not proven yet** — specified, no test behind it

`R08.12` Wastage is recorded, not absorbed

- **When** consumption exceeds the BOM
- **Then** the excess is recorded as wastage against the order with its reason
- **Never** quietly increasing the BOM to match what was used, which destroys the only signal that something is going wrong
- **Not proven yet** — specified, no test behind it

`R08.13` A stage cannot be skipped without being recorded as skipped

- **When** work moves past a defined stage without that stage being marked
- **Then** the skip is recorded on the order
- **Never** letting the stage silently complete, which makes every stage-time figure fiction
- **Not proven yet** — specified, no test behind it

`R08.14` An advance to a worker is a balance, not a deduction from nowhere

- **When** an advance is paid
- **Then** it is held against that worker and recovered from later payouts, with the running balance visible
- **Never** deducting an amount at payout time that cannot be traced to a specific advance
- **Not proven yet** — specified, no test behind it

`R08.15` A rework carries the cost of the rework

- **When** a piece is returned to a stage to be redone
- **Then** the additional labour is costed to the design that caused it

- **Never** costing it as new production, which makes a failing design look as profitable as a good one
- **Not proven yet** — specified, no test behind it

`R08.16` Material consumed is the average per piece times the pieces made

- **When** consumption is costed against a production run
- **Then** consumption equals the average consumption per piece × pieces produced, and the difference against the bill of materials is recorded as wastage
- **Never** back-fitting the average to whatever was issued, which makes wastage mathematically impossible to see
- **Not proven yet** — specified, no test behind it

`R08.17` A set type comes from the rate master, and an inferred one says so

- **When** a design is classified into a set type
- **Then** the rate master's Set column decides it; when the design is absent, the type is inferred from which component columns actually carry pieces and the design is flagged as inferred
- **Never** presenting an inferred classification as though it came from the master
- **Proven** by `brand/suite/studio/verify_studio.js › the two-row heading is read, so three same-named columns stay three components`

`R08.18` An alteration caused by the worker's own mistake is unpaid

- **When** a piece is reworked because of an error by the person who made it
- **Then** the alteration hours are recorded and paid at zero
- **Never** paying for the rework at the standard alteration rate, and never leaving the hours unrecorded — the time still happened and the design still bore the cost
- **Not proven yet** — specified, no test behind it

`R08.19` Alteration time is paid at the alteration rate, not the piece rate

- **When** admin-assigned alteration hours are settled
- **Then** they are paid at the hourly alteration rate in force and added to that worker's payout
- **Never** folding alteration hours into the piece count, which corrupts both the production figure and the earnings figure at once
- **Not proven yet** — specified, no test behind it

`R08.20` A contract worker paid by the hour has no attendance row

- **When** a contract role is settled
- **Then** payment is hours worked × the agreed hourly rate, recorded against the person without an attendance record
- **Never** forcing a contract worker through the salaried attendance model, which produces a monthly figure nobody agreed to
- **Not proven yet** — specified, no test behind it

Module 09 · Quality & Compliance

Certify what was received and what was made

Quality inspection used to live buried as one step inside Manufacturing; it stands on its own here because a rejection at goods-receipt and a rejection on the production floor are the same discipline, and because the certificates a business holds — the proof it follows a standard — belong next to the inspections that back them up, not scattered across email.

Reads from	Purchase, Manufacturing
Writes to	Purchase, Manufacturing, Inventory & Catalog
Apps	2
Rules	7, of which 0 are proven by a test that runs

The 2 apps in this module

Quality Control — SPECIFIED — designed, not built

Accept, reject or send for rework, with reasons that feed the mill's accept rate and the karigar's record.

Certificate & Compliance Register — SPECIFIED — designed, not built

Every standard the business or a vendor holds — a safety, labour or environmental certification — with its issue date, its expiry, and the audit that backs it, so a certificate about to lapse is visible before a buyer asks for it and finds it already expired. What this register tracks is what a sustainability report downstream is actually built from.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Checking · lot 8841 · before it is packed

- **Figures across the top** — Lots checked today 22 · Failed 3 · Accept rate 94.6% · Alter queue 48
- **Columns** — Lot · Design · Checked for · Result
- **Rows** — 4 worked examples, the first reading: 8841 · V-1180 Anarkali · Stitch, measurement, finish · Pass
- **Controls** — Failures blocking dispatch · Lots packed with an open failure

The 7 rules this module must satisfy

`R09.1` A failed check blocks the next stage

- **When** an inspection fails
- **Then** the batch cannot progress until it is passed, reworked or written off
- **Never** letting it move with the failure noted, which sends a known defect to a customer

- **Not proven yet** — specified, no test behind it

`R09.2` A check names the person who did it

- **When** any inspection is recorded
- **Then** the inspector, the time and the sample size are stored
- **Never** accepting an anonymous pass, which cannot be investigated when the complaints arrive
- **Not proven yet** — specified, no test behind it

`R09.3` An expiring certificate warns before it expires

- **When** a certificate approaches its expiry
- **Then** it is raised while there is still time to renew
- **Never** discovering the lapse at the moment a buyer asks for it
- **Not proven yet** — specified, no test behind it

`R09.4` A rejected batch cannot be sold as first quality

- **When** a batch is rejected
- **Then** it is marked and can only be sold through a channel that accepts seconds
- **Never** letting it re-enter the ordinary sellable pool
- **Not proven yet** — specified, no test behind it

`R09.5` A defect is attached to the design and the stage

- **When** a defect is recorded
- **Then** both the design and the stage that produced it are named
- **Never** recording it against the batch alone, which loses the pattern that would have prevented the next one
- **Not proven yet** — specified, no test behind it

`R09.6` A compliance document is evidence, not a checkbox

- **When** a compliance requirement is marked met
- **Then** the document proving it is attached
- **Never** accepting a tick with nothing behind it, which is what fails an audit
- **Not proven yet** — specified, no test behind it

`R09.7` A sustainability figure comes from the same evidence

- **When** an ESG figure is reported
- **Then** it is computed from the certificate and audit records already on file
- **Never** assembling it separately once a year from numbers nobody can trace
- **Not proven yet** — specified, no test behind it

Module 10 · Warehouse

Pick the right design first time — and prove what you sent

Bin-level instructions and barcode scanning so the right piece leaves the godown and stock stays honest — and a recording of each parcel being packed, because a wrong-return claim is settled by footage, not by argument.

Reads from	Sales, E-commerce / OMS, Inventory & Catalog
Writes to	Inventory & Catalog, Sales, E-commerce / OMS
Apps	3
Rules	8, of which 0 are proven by a test that runs

The 3 apps in this module

Picking & Bins — SPECIFIED — designed, not built

Pick lists in walking order through the godown, by design and size, so nobody crosses the floor twice.

Barcode Operations — SPECIFIED — designed, not built

Scan to pick, pack, dispatch and run a physical stock count from a phone — the same scan whether the order came from a marketplace, your Shopify site or the counter.

Packing Video — SPECIFIED — designed, not built

Every parcel filmed as it is packed and indexed by its order number, so when a panel says the wrong piece was sent, the clip goes into the claim.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Pick wave 22 · godown zone A → C

- **Figures across the top** — Lines to pick 486 · Pickers on floor 6 · Short-picked 4 · Packed & filmed 241
- **Columns** — Bin · Design & size · Qty · Picker · State
- **Rows** — 4 worked examples, the first reading: A-04-2 · VG-1180 Teal · M · 24 · Ravi · Picked
- **Controls** — Picked first time right · Parcels with footage

The 8 rules this module must satisfy

`R10.1` A pick is confirmed against the bin it came from

- **When** an item is picked
- **Then** the bin is recorded on the movement
- **Never** decrementing a warehouse total with no bin, which makes the next cycle count unexplainable

- **Not proven yet** — specified, no test behind it

`R10.2` A short pick stops the pack, it does not silently reduce the order

- **When** the picker cannot find the full quantity
- **Then** the shortage is raised against the order and the pack waits
- **Never** packing what was found and invoicing for it as though that was the order
- **Not proven yet** — specified, no test behind it

`R10.3` A scan is the same event as a keyed entry

- **When** a code is captured by scanner, phone camera or typing
- **Then** the same movement is written
- **Never** having a scanning path that writes different records from the manual path
- **Not proven yet** — specified, no test behind it

`R10.4` A cycle count adjustment names a reason

- **When** a count differs from the system
- **Then** the adjustment records the reason and the person
- **Never** writing the system down to the counted figure with no explanation, which hides theft and damage equally well
- **Not proven yet** — specified, no test behind it

`R10.5` The packing video is linked to the shipment

- **When** a parcel is recorded on video at packing
- **Then** the recording is attached to that shipment
- **Never** keeping the footage in a folder by date, which makes it unusable in the dispute it exists for
- **Not proven yet** — specified, no test behind it

`R10.6` A bin holds a location, not a guess

- **When** stock is put away
- **Then** the destination bin is captured at put-away
- **Never** assigning a default bin so the step can be skipped
- **Not proven yet** — specified, no test behind it

`R10.7` A dispatch cut-off is per channel

- **When** a channel has a handover deadline
- **Then** the queue is ordered and warned against that channel’s own cut-off
- **Never** applying one cut-off to all of them, which misses the earliest and idles for the latest
- **Not proven yet** — specified, no test behind it

`R10.8` A returned parcel is inspected before it is anything else

- **When** a return arrives at the warehouse

- **Then** it is booked into a return-inspection location first
- **Never** restocking on arrival, which puts an unchecked item back on sale
- **Not proven yet** — specified, no test behind it

Module 11 · Logistics

The courier network — rates, failed deliveries and the COD money

Booking one parcel happens on the order. This module is the network behind it: what Delhivery, Blue Dart and the rest charge to that pin code before you pick one, what happens to a delivery that fails in a small town, and whether the cash collected at the door reached your bank.

Reads from	Sales, E-commerce / OMS, Warehouse
Writes to	Accounting & GST, Sales, E-commerce / OMS
Apps	5
Rules	11, of which 0 are proven by a test that runs

The 5 apps in this module

Rates & Zones — SPECIFIED — designed, not built

Every courier’s rate card by zone, weight slab and service — so the cheapest and the fastest option for this parcel are both known before it is booked.

NDR & RTO Rescue — SPECIFIED — designed, not built

A failed delivery worked while it can still be saved — reattempt, call, correct the address — before it becomes a return you pay for twice.

COD Remittance — SPECIFIED — designed, not built

What the courier collected at the door against what reached the Surat account, parcel by parcel, with every shortfall named and aged.

Handover & Manifest — SPECIFIED — designed, not built

What is expected out today against what the pickup boy actually took, per courier and per service. The manifest to hand him, the one-time code to confirm it, and a signed note of what was left behind — so a parcel lost between the packing table and the van has an owner.

Fleet — SPECIFIED — designed, not built

Your own delivery vehicles, if you run any — what each trip cost, what is due for service, and what that adds to freight. Optional; most businesses run couriers only.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Handover · what went out against what they took

- **Figures across the top** — Expected out 1,046 · Handed over 1,012 · Left behind 34 · COD not remitted ₹3.8 L
- **Columns** — Courier · service · Expected · Handed over · Left · Code
- **Rows** — 4 worked examples, the first reading: Courier A · large · 126 · 126 · 0 · confirmed
- **Controls** — Manifests signed the same day · Parcels traced after handover

The 11 rules this module must satisfy

`R11.1` The courier rate is checked against the packed weight

- **When** a courier bills for a shipment
- **Then** the billed weight is compared with the packed weight recorded at packing
- **Never** accepting the courier’s weight without comparison, which is the most consistently overcharged line in the business
- **Not proven yet** — specified, no test behind it

`R11.2` A weight dispute is raised with the evidence attached

- **When** billed and packed weight differ beyond tolerance
- **Then** a dispute is raised carrying the packing record
- **Never** absorbing the difference because each one is small
- **Not proven yet** — specified, no test behind it

`R11.3` An undelivered parcel is chased before it becomes a return

- **When** a delivery attempt fails
- **Then** the NDR is actioned within the window the courier allows
- **Never** letting it lapse into a return, which costs the freight twice and the sale once
- **Not proven yet** — specified, no test behind it

`R11.4` COD collected is a receivable until it is remitted

- **When** a COD parcel is delivered
- **Then** the amount is a receivable from the courier
- **Never** treating delivery as payment, which reports cash the business does not have
- **Not proven yet** — specified, no test behind it

`R11.5` A remittance is matched parcel by parcel

- **When** a courier remits COD
- **Then** each parcel in the remittance is matched individually

- **Never** accepting the total, which is how short remittances go unnoticed for months
- **Not proven yet** — specified, no test behind it

`R11.6` A manifest is a record, not a printout

- **When** parcels are handed over
- **Then** the handover is recorded against each shipment with the time and the person
- **Never** keeping only a signed sheet, which cannot be queried when a parcel is disputed
- **Not proven yet** — specified, no test behind it

`R11.7` An RTO parcel is stock again only after inspection

- **When** a return to origin is received
- **Then** it goes through inspection before it can be sold
- **Never** restocking it automatically on scan
- **Not proven yet** — specified, no test behind it

`R11.8` Freight cost reaches the order it belongs to

- **When** a shipment is costed
- **Then** the freight is attributed to the order
- **Never** holding freight only as a monthly expense, which makes per-order and per-channel profit fiction
- **Not proven yet** — specified, no test behind it

`R11.9` A courier can be changed without losing history

- **When** a courier is switched off
- **Then** every past shipment, AWB and dispute stays readable
- **Never** making history depend on an integration that is still connected
- **Not proven yet** — specified, no test behind it

`R11.10` A zone and rate card are dated

- **When** courier rates change
- **Then** the new card is stored with its effective date
- **Never** overwriting the card, which makes every past shipment look mischarged
- **Not proven yet** — specified, no test behind it

`R11.11` A partial-COD order has two collections and both are tracked

- **When** an order is placed with an advance online and the balance on delivery
- **Then** the advance is a receipt now and the balance is a receivable from the courier until it is remitted
- **Never** treating the advance as the whole payment, which makes every such order look settled while most of the money is still outstanding
- **Not proven yet** — specified, no test behind it

Module 12 · Accounting & GST

Books that always balance — and no BUSY needed

A full double-entry ledger built for Indian compliance, keeping the books itself. B2B sales, returns, mill purchases, payments and receipts are entered by hand because a person decides them; every website, marketplace and counter sale posts itself.

Reads from	Every module
Writes to	Finance Reports, Treasury & Financial Planning
Apps	9
Rules	24, of which 16 are proven by a test that runs

The 9 apps in this module

Accounting — SPECIFIED — designed, not built

Double-entry books where every voucher balances and the trial balance always ties.

Invoicing — SPECIFIED — designed, not built

GST tax invoices and receipts, worked out from the lines to the paise. Where a panel raises its own invoice you keep both numbers on the order — theirs and your own series — so the panel's paperwork and your books point at the same sale.

Expenses — SPECIFIED — designed, not built

Spend captured by category with approvals, and bill OCR to save typing.

GST & Tax — SPECIFIED — designed, not built

CGST, SGST, IGST, TDS, TCS, input credit, GSTR-1 and GSTR-3B — filed per registration, so a group where one company is registered and another is not files exactly what each one owes and nothing it does not.

ITC Reconciliation — SPECIFIED — designed, not built

Every input credit matched against the government's own GSTR-2A/2B before a return is filed, so a credit claimed is a credit that is actually on record.

Receivables, Payables & PDC — SPECIFIED — designed, not built

Payments and receipts allocated against named open invoices, and a register for post-dated cheques that posts only on the date they are realised, not the date they were written.

Fixed Assets & Depreciation — SPECIFIED — designed, not built

The asset register with both Straight-Line and Written-Down-Value depreciation tracked side by side, and disposal posting its gain or loss straight to the books.

Year-End Close & Period Lock — SPECIFIED — designed, not built

Profit-and-loss accounts reset and balance-sheet accounts carry forward at year end, and a reviewed period locks against backdated edits until an admin unlocks it — an act that is itself logged.

Finance Reports — SPECIFIED — designed, not built

P&L, balance sheet, and profit by channel, design and SKU — so you know which anarkali actually earned money after commission, shipping and returns.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Trial balance · it always ties

- **Figures across the top** — Revenue YTD ₹11.8 Cr · Gross margin 38.2% · GST payable ₹4.1 L · ITC available ₹3.6 L
- **Columns** — Head · Debit · Credit · This month
- **Rows** — 4 worked examples, the first reading: Sales · — · ₹1,42,08,400 · +12%
- **Controls** — GSTR-1 lines matched · Vouchers posted automatically

The 24 rules this module must satisfy

`R12.1` Money is an integer count of paise

- **When** any amount is held, added or compared
- **Then** it is an integer number of paise, becoming a decimal string only where a person reads it
- **Never** holding money in a floating-point number, where ₹0.10 + ₹0.20 is not ₹0.30 and a trial balance stops balancing
- **Proven** by `core/tests/core.test.js › the classic float error cannot happen here`

`R12.2` An amount finer than a paisa is refused, not rounded

- **When** a computation produces a fraction of a paisa
- **Then** it is refused and the caller must round deliberately
- **Never** rounding silently, which is how two sides of the same figure drift apart and nobody can say which is right
- **Proven** by `core/tests/core.test.js › an amount finer than a paisa is refused rather than silently rounded`

`R12.3` A split sums back to the original, exactly

- **When** an amount is divided — across lines, across companies, across periods
- **Then** the parts add back to the whole, with the round-off returned as its own figure
- **Never** losing or inventing a paisa in the split, and never hiding the remainder inside the largest part
- **Proven** by `core/tests/core.test.js › a split always sums back to the original — no paisa lost or invented`

`R12.4` An unbalanced entry is refused, with the gap named

- **When** a voucher is posted whose debits and credits differ
- **Then** it is refused and the difference is stated
- **Never** posting it to a suspense account to make it balance, which converts an error into a permanent record
- **Proven** by `core/tests/core.test.js › an unbalanced entry is refused, with the gap named`

`R12.5` A line cannot be a debit and a credit at once

- **When** a posting line carries both
- **Then** it is refused
- **Never** netting the two into whichever is larger
- **Proven** by `core/tests/core.test.js › a line cannot be a debit and a credit at once`

`R12.6` The trial balance is computed, never stored

- **When** the trial balance is asked for
- **Then** it is summed from the posting lines at that moment
- **Never** reading a maintained total, which is a number that can be wrong without anything looking wrong
- **Proven** by `core/tests/core.test.js › the trial balance is computed from the lines, never stored`

`R12.7` A locked period refuses a backdated entry

- **When** a voucher is dated inside a closed period
- **Then** it is refused and the lock that stopped it is named
- **Never** posting it into the current period instead, which silently moves last year’s result into this one
- **Proven** by `core/tests/core.test.js › a locked period refuses a backdated entry`

`R12.8` Unlocking a period is itself recorded

- **When** a closed period is reopened
- **Then** who reopened it, when and why is written to the trail
- **Never** allowing a quiet reopen, which is the one action that could undo every other guarantee here
- **Proven** by `core/tests/core.test.js › unlocking a period is itself recorded`

`R12.9` A tax rate resolves on the date of the document

- **When** tax is computed for any invoice
- **Then** the rate in force on that document’s date is used
- **Never** applying today’s rate to an old invoice, which makes correct history look like an error
- **Proven** by `core/tests/core.test.js › a tax rate resolves on a date, so old invoices stay correct`

`R12.10` Two rates covering one date is ambiguous, not a coin toss

- **When** two effective-dated rows overlap for the same date
- **Then** the resolution is refused and the overlap is named
- **Never** picking the newer one, which makes the answer depend on insertion order

- **Proven** by `core/tests/core.test.js › two rows covering one month is ambiguous, not a coin toss`

`R12.11` A voided entry is reversed, never erased

- **When** a posted voucher is wrong
- **Then** a reversing entry is posted and both stay visible
- **Never** editing or deleting the original, which is the difference between a correction and a cover-up
- **Proven** by `core/tests/core.test.js › voiding is the only removal, and it is reversible`

`R12.12` Every figure clicks down to the record that produced it

- **When** any total appears on any screen
- **Then** it is a live query that can be opened down to its vouchers and their documents
- **Never** showing a figure that cannot be traced — an untraceable number is a defect, not a rounding difference
- **Proven** by `core/tests/core.test.js › the trial balance is computed from the lines, never stored`

`R12.13` An invoice number is sequential per company and per series

- **When** an invoice is raised
- **Then** it takes the next number in that company's series
- **Never** reusing, skipping or back-filling a number, which is the first thing a tax audit tests
- **Not proven yet** — specified, no test behind it

`R12.14` A GST return is built from vouchers, not from a summary

- **When** GSTR-1 or 3B is prepared
- **Then** it is computed from the underlying invoices
- **Never** accepting a typed summary figure, which cannot be reconciled when the portal disagrees
- **Not proven yet** — specified, no test behind it

`R12.15` ITC is claimed only where the supplier has filed

- **When** input credit is taken
- **Then** it is matched against the supplier's filed data and the unmatched part is held
- **Never** claiming everything and reversing later, which turns a reconciliation into a liability
- **Not proven yet** — specified, no test behind it

`R12.16` A place of supply decides the tax, not the billing address

- **When** GST is computed
- **Then** the place of supply determines CGST/SGST or IGST
- **Never** defaulting to the billing address, which mis-splits the tax on every drop-ship
- **Not proven yet** — specified, no test behind it

`R12.17` A credit note references the invoice it reverses

- **When** a credit note is raised

- **Then** the original invoice is named on it
- **Never** issuing a free-standing credit note, which cannot be matched in either set of books
- **Not proven yet** — specified, no test behind it

`R12.18` Depreciation is posted, not just calculated

- **When** a period closes
- **Then** depreciation is posted as an entry like any other
- **Never** showing it as a computed figure on a report while the ledger disagrees
- **Not proven yet** — specified, no test behind it

`R12.19` A company with no tax registration is still a company

- **When** a group company has no registration of its own
- **Then** it keeps its own books and joins the group figures
- **Never** dragging it into a return it does not belong in, and never leaving it out of the group result
- **Not proven yet** — specified, no test behind it

`R12.20` Year-end close locks, and the lock is the record

- **When** a financial year is closed
- **Then** the period is locked and the closing balances are carried forward as an entry
- **Never** leaving the year open indefinitely so late entries can drift in unnoticed
- **Proven** by `core/tests/core.test.js › a locked period refuses a backdated entry`

`R12.21` Every voucher type posts through one engine

- **When** a sale, purchase, credit note, debit note, payment, receipt, journal, contra or counter sale is recorded
- **Then** all nine post through the same ledger routine
- **Never** giving a voucher type its own posting logic — this is where home-built accounting breaks and the modules stop agreeing about the same figure
- **Proven** by `core/tests/core.test.js › a balanced entry posts`

`R12.22` Net GST is input against output, per period, per company

- **When** the GST position for a period is computed
- **Then** it is output tax less eligible input credit for that company and that period
- **Never** netting across companies, which offsets one registration’s liability with another’s credit and is not a return anyone may file
- **Not proven yet** — specified, no test behind it

`R12.23` Money never becomes a float, in any layer

- **When** an amount is stored, moved between the engine and the database, or exported
- **Then** it stays an integer count of paise end to end, converted for display only
- **Never** a real, double, float or an unlabelled decimal column anywhere a money value lives

- **Proven** by `core/tests/schema.test.js › no money column is a float, in either schema`

`R12.24` A money column says what unit it is in

- **When** a column holds an amount
- **Then** its name ends in paise
- **Never** a column called total, amount or cost with no unit — the same name read as rupees by one developer and paise by the next is a factor of a hundred in the books
- **Proven** by `core/tests/schema.test.js › no column is named amount/price/cost without saying what unit it is in`

Module 13 · Treasury & Financial Planning

Know what cash is coming, not just what already arrived

Accounting records what happened; this module is concerned with what happens next — how much cash is actually expected, when, and whether spend against a budget is on track before the month closes and turns the answer into history.

Reads from	Accounting & GST, Sales, Purchase
Writes to	Accounting & GST
Apps	3
Rules	8, of which 1 are proven by a test that runs

The 3 apps in this module

Cash Flow Forecast — SPECIFIED — designed, not built

Expected receipts and expected payments laid out by week, drawn from open invoices and open bills rather than typed in by hand — so a cash shortfall is visible weeks before the date it would actually bite.

Banking & Reconciliation — SPECIFIED — designed, not built

Bank statement lines matched against the ledger’s own record of what should have moved, with anything unmatched surfaced instead of silently carried forward.

Budget vs Actual — SPECIFIED — designed, not built

A budget set per category and period, and actual spend from Accounting tracked against it as the period runs — not only after it closes, when the only thing left to do is explain the variance.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Cash position · next 14 days

- **Figures across the top** — Cash + bank ₹38.4 L · Due out ₹44.2 L · Expected in ₹52.6 L · Tightest day Day 8
- **Columns** — When · Out · In · Running
- **Rows** — 4 worked examples, the first reading: This week · mill payments · ₹18.4 L · ₹21.2 L · ₹41.2 L
- **Controls** — Forecast lines with a named assumption · Projections posted to the ledger

The 8 rules this module must satisfy

`R13.1` A forecast never posts to the ledger

- **When** a cash-flow projection is produced
- **Then** it is held as a projection, separate from posted entries
- **Never** writing an expected receipt into the books, which reports money that has not arrived
- **Not proven yet** — specified, no test behind it

`R13.2` A bank line is matched to a voucher, not to a total

- **When** a bank statement is reconciled
- **Then** each line is matched to the entry that caused it
- **Never** reconciling on the closing balance alone, which hides two errors that happen to cancel
- **Not proven yet** — specified, no test behind it

`R13.3` An unmatched bank line stays visible until it is explained

- **When** a statement line cannot be matched
- **Then** it stays on the unreconciled list with its age
- **Never** writing it off to a sundry account to clear the screen
- **Not proven yet** — specified, no test behind it

`R13.4` A PDC is a commitment before it is cash

- **When** a post-dated cheque is received
- **Then** it is tracked as a commitment until it clears
- **Never** recognising it as cash on receipt
- **Not proven yet** — specified, no test behind it

`R13.5` Budget versus actual compares like with like

- **When** a variance is shown
- **Then** both sides use the same period, company and account basis
- **Never** comparing a full-year budget against a part-year actual without saying so

- **Not proven yet** — specified, no test behind it

`R13.6` A cash forecast names its assumptions

- **When** a projection is produced
- **Then** the collection and payment assumptions behind it are stored with it
- **Never** presenting a projection whose basis cannot be recovered a month later
- **Not proven yet** — specified, no test behind it

`R13.7` Inter-company funding is recorded on both sides

- **When** one group company funds another
- **Then** both companies post it, naming each other as counterparty
- **Never** recording it in one set of books only, which leaves the group permanently out of balance
- **Proven** by `core/tests/core.test.js › an entry cannot be its own counterparty`

`R13.8` A currency amount keeps the rate it was converted at

- **When** a foreign-currency transaction is recorded
- **Then** the original amount, the currency and the rate used are all stored
- **Never** storing only the converted figure, which cannot be revalued or explained afterwards
- **Not proven yet** — specified, no test behind it

Module 14 · Settlement

Get paid what the panels owe you — cycle by cycle

Matching one payout to one order line happens in OMS. This is the level above: the settlement cycle each panel runs, the commission it actually charged against the rate card it published, and the TCS it deducted in your name.

Reads from	E-commerce / OMS, Accounting & GST
Writes to	Accounting & GST
Apps	3
Rules	13, of which 0 are proven by a test that runs

The 3 apps in this module

Payout Cycles — SPECIFIED — designed, not built

Every settlement cycle each panel runs — what it should pay, what actually landed in the bank, and on which day — so a late payout is visible the day it is late, not at month end.

Fee & Commission Audit — SPECIFIED — designed, not built

The commission a panel publishes for a category against what it actually took, style by style. A quiet rate change is caught the first time it is applied, not at year end — and your seller tier sits on the same screen, because the tier is what the rate card hangs off, and slipping out of one quietly costs more than any single deduction.

TCS & TDS Register — SPECIFIED — designed, not built

Every rupee the panels deducted as TCS, and TDS on job work, matched against the portal's own figures — so the credit you claim is the credit you are owed.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Settlement cycles · what each panel really paid

- **Figures across the top** — Due this cycle ₹52.4 L · Actually paid ₹49.1 L · Gap ₹3.3 L · Claims filed 18
- **Columns** — Panel · Cycle · Should pay · Paid · Gap
- **Rows** — 4 worked examples, the first reading: Myntra · 12–18 Jul · ₹18,40,000 · ₹18,40,000 · ₹0
- **Controls** — Commission charged as published · Claims recovered

The 13 rules this module must satisfy

`R14.1` A payout is matched line by line to orders

- **When** a marketplace settlement file arrives
- **Then** every line is matched to the order it belongs to
- **Never** accepting the net credited amount, which is how a short payment becomes invisible
- **Not proven yet** — specified, no test behind it

`R14.2` Every deduction is identified before the payout is accepted

- **When** commission, shipping, penalty, TCS or TDS is deducted
- **Then** each is posted to its own account
- **Never** posting the deductions as one lump, which makes an overcharge impossible to find
- **Not proven yet** — specified, no test behind it

`R14.3` A variance beyond tolerance raises a claim

- **When** the settled amount differs from the expected amount
- **Then** a claim is raised carrying the order, the expectation and the difference
- **Never** absorbing it because it is small — the small ones are the recurring ones
- **Not proven yet** — specified, no test behind it

`R14.4` A claim has a deadline and the deadline is tracked

- **When** a claim is raised
- **Then** the channel's filing window is stored and warned on

- **Never** letting a valid claim expire unfiled
- **Not proven yet** — specified, no test behind it

`R14.5` An expected settlement exists from the moment of the sale

- **When** an order is confirmed on a marketplace
- **Then** a settlement expectation is created then
- **Never** waiting for the payout to discover what should have arrived
- **Not proven yet** — specified, no test behind it

`R14.6` TCS and TDS are receivables, not costs

- **When** a marketplace deducts tax at source
- **Then** it is posted as a receivable against the tax authority
- **Never** expensing it, which understates profit and loses the credit
- **Not proven yet** — specified, no test behind it

`R14.7` A settlement is reconciled to the bank, not just to the file

- **When** a payout is recorded
- **Then** it is matched to the actual bank credit
- **Never** treating the settlement report as proof that the money arrived
- **Not proven yet** — specified, no test behind it

`R14.8` A re-sent settlement file does not double-post

- **When** the same settlement file is imported twice
- **Then** already-matched lines are recognised and skipped
- **Never** posting them again, which doubles both revenue and deductions
- **Not proven yet** — specified, no test behind it

`R14.9` A fee schedule is dated and compared against

- **When** a commission is deducted
- **Then** it is checked against the agreed rate in force on that date
- **Never** accepting whatever rate the file states, which is the single largest silent leak in marketplace trade
- **Not proven yet** — specified, no test behind it

`R14.10` A settled order is profitable or unprofitable at the SKU

- **When** a payout is fully matched
- **Then** the true net per SKU is computed after every deduction
- **Never** judging profitability on the listed price, which ignores the third of it that never arrives
- **Not proven yet** — specified, no test behind it

`R14.11` A claim that is paid closes against the original variance

- **When** a channel credits a claim
- **Then** it is matched back to the variance it settles
- **Never** posting the credit as unrelated income, which leaves the variance open forever
- **Not proven yet** — specified, no test behind it

`R14.12` A settlement figure never overwrites a sale figure

- **When** the settlement disagrees with the order
- **Then** both are kept and the difference is the variance
- **Never** adjusting the original sale to match the payout, which erases the evidence of the shortfall
- **Not proven yet** — specified, no test behind it

`R14.13` The realisation on a marketplace sale is the price minus every deduction

- **When** what a channel sale actually earned is computed
- **Then** it is the selling price less shipping, commission, fixed fee, GST on those fees, TCS and TDS — each taken from the settlement file
- **Never** judging a sale on its listed price, which ignores the part of it that never arrives, and never applying an assumed commission percentage when the file states the real one
- **Not proven yet** — specified, no test behind it

Module 15 · E-commerce / OMS

Seven panels, one queue — and every rupee accounted for

Stop logging into Myntra, then Flipkart, then Ajio. Every marketplace order lands in one pipeline and your stock goes out to all of them — then the money side closes out in the same module: what the panel paid, what it kept as commission, what came back, and what it still owes you.

Reads from	Inventory & Catalog, CRM, Sales, Accounting & GST, Logistics, Settlement
Writes to	Inventory & Catalog, Accounting & GST, Warehouse, Logistics, Settlement
Apps	11
Rules	19, of which 10 are proven by a test that runs

The 11 apps in this module

Marketplace OMS — **BROWSER APP** — opens and self-tests, no shared database behind it

Myntra, Flipkart, Ajio, Amazon, Meesho, Nykaa and JioMart in a single queue — processed all together, channel-wise, or design-wise. The stages the panels really use, with the right cut-off counting down on each order — a

quick-commerce or air-shipped order is not due at the same hour as a standard one. Priority orders at the top, and the day grouped by design so a Muskan Purple is picked once for eleven parcels instead of eleven times.

Order Management — **BROWSER APP** — opens and self-tests, no shared database behind it

One pipeline from new to delivered, whether the order came from a seller panel, your Shopify or WooCommerce site, a dealer or the counter.

Manual Data Check — SPECIFIED — designed, not built

The order and return sheets you already download from the panels, and the offline registers from the three shops — one file or a whole ZIP — read back as ten cross-checks: net sale after commission and fees, month, design, state, wrong returns, SPF claims, ads, payouts and GST. Every figure clicks through to the transactions behind it.

Reconciliation — SPECIFIED — designed, not built

Match every marketplace payout to the order line that earned it, and expose the gap.

Claims & Disputes — SPECIFIED — designed, not built

Weight disputes, SPF shortfalls, parcels lost in transit and returns that came back with a different piece inside — filed as claims with the packing footage attached, and answered before they close. A claim awaiting your reply is money; one closed for no response is nothing, so the days left sit beside the amount.

Returns / RMA — SPECIFIED — designed, not built

Customer returns, courier returns and wrong returns kept apart — because only one of the three is really your fault, and only one of them turns into dead stock.

Channels & Storefronts — SPECIFIED — designed, not built

Connect a channel once and it stays in step: catalogue out, price out, stock out, orders in. Seven marketplaces and any website platform — Shopify, WooCommerce, Magento, BigCommerce, Wix or a custom site over its own API — each switchable without touching your data. Shopsy and any other storefront a channel runs alongside its main one counts as its own channel here. Where a channel has no open interface, its own downloaded report is a first-class way in. A channel may also know you by a different trading name — that is a label on the channel, not a second company, so it tags the order and the payout without ever splitting your books.

Labels & Documents — SPECIFIED — designed, not built

The panel gives you a PDF; this hands the packing table something it can work from. Cropped to 4×6 for every channel, your design code printed large where the panel left it off, the invoice and slip merged behind it, and the whole batch to the label printer in one job. Reprint one parcel without redoing the lot — and no customer’s name and address is ever uploaded to an outside website to be cropped.

Listing & Catalog Manager — SPECIFIED — designed, not built

Bulk-create and bulk-edit listings across every channel from the one product record in Inventory & Catalog, and catch the mismatches that quietly cost sales: listed but out of stock, or in stock but never listed.

Size / Fit Recommendation AI — SPECIFIED — designed, not built

A fit suggestion at the point of purchase, built from the item’s own measurements and the return history of buyers who picked each size — aimed straight at the return reason that costs the most: the right item in the wrong size.

AR / Virtual Try-On — SPECIFIED — designed, not built

A way to see drape, fit and colour on a screen before buying, for items where a flat photo alone leaves too much to guess.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Panel queue · 9 channels · cut-off running

- **Figures across the top** — To accept 62 · To pack 214 · Cut-off in 2 h 9 · Handed over today 188
- **Columns** — Panel · To accept · To pack · RTD · Cut-off
- **Rows** — 5 worked examples, the first reading: Myntra · 18 · 62 · 48 · 1 PM
- **Controls** — Dispatched inside the cut-off · Labels printed straight from here

The 19 rules this module must satisfy

`R15.1` Companies and channels are read from the data, never from a list in the code

- **When** orders or sheets from any number of companies and channels are processed
- **Then** the companies and channels present are discovered and each gets its own columns
- **Never** writing a fixed set of companies or channels into the software, which caps the business at whatever it happened to have on the day the code was written
- **Proven** by `brand/suite/studio/verify_studio.js › the companies are found from the sheets, not from a hardcoded list`

`R15.2` A tenth or eleventh channel needs no code change

- **When** a new marketplace or company is added
- **Then** it is a row, and every figure, column and consolidation follows
- **Never** requiring a release to sell somewhere new
- **Proven** by `core/tests/core.test.js › an eleventh company and an eleventh channel need no code change`

`R15.3` A channel belongs to a company

- **When** two companies both sell on the same marketplace
- **Then** each has its own channel record, and both may use the same short code
- **Never** sharing one channel across companies, which merges two companies' sales into one figure
- **Proven** by `core/tests/core.test.js › a channel belongs to a company — two companies may both call one AMZN`

`R15.4` A price is never invented for an item that has none

- **When** an item has no price on file
- **Then** it is reported as having no price and named in the summary

- **Never** substituting an average or a similar item's price, which quietly fabricates revenue
- **Proven** by `brand/suite/studio/verify_studio.js › the price status matches, and no price was ever invented`

`R15.5` Net is sale minus return, and inventory is net plus wrong return

- **When** quantities are rolled up
- **Then** net sale is sale minus return, and the inventory figure adds back the wrong returns
- **Never** treating a wrong return as ordinary saleable stock, because it is not the item that was sent out
- **Proven** by `brand/suite/studio/verify_studio.js › sale minus return is the net, and net plus wrong return is the inventory`

`R15.6` A blank cell is blank, not a value

- **When** a column contains only whitespace
- **Then** it is read as empty
- **Never** treating a lone space as a marked entry, which converts formatting into business fact
- **Proven** by `brand/suite/studio/verify_studio.js › a lone space in the Wrong Return column is not a wrong return`

`R15.7` An item that only ever came back is still reported

- **When** an item has returns but no sales in the period
- **Then** it appears with its returns
- **Never** dropping it because it has no sale line, which hides the worst-performing items entirely
- **Proven** by `brand/suite/studio/verify_studio.js › an item that only ever came back is still reported`

`R15.8` A totals row is the sum of the rows above it

- **When** a report shows a total
- **Then** it equals the rows it sits under
- **Never** computing the total by a different route from the detail, which is how a report disagrees with itself
- **Proven** by `brand/suite/studio/verify_studio.js › the totals row is the sum of the rows above it`

`R15.9` A marketplace order pull creates a real order

- **When** orders are fetched from a channel
- **Then** a sales order is created, stock is reserved, and the pick list follows
- **Never** holding channel orders in a staging area that has to be re-entered to become real
- **Not proven yet** — specified, no test behind it

`R15.10` A cancelled channel order releases its reservation

- **When** the channel cancels an order
- **Then** the reservation is released and the cancellation recorded
- **Never** leaving stock reserved against an order the channel has already dropped

- **Not proven yet** — specified, no test behind it

`R15.11` A wrong return is dead stock, not stock

- **When** a return is inspected and found to be a different or damaged item
- **Then** it is written to dead stock with its cost recognised as a loss
- **Never** restocking it as first quality, which sells a customer the same problem twice
- **Not proven yet** — specified, no test behind it

`R15.12` A listing rejected by a channel says why

- **When** a push to a channel fails
- **Then** the rejection and its reason are reported back against the listing
- **Never** reporting a push as successful when part of it failed, which leaves the business believing it is present where it is not
- **Not proven yet** — specified, no test behind it

`R15.13` A manual data check is a recorded step, not a habit

- **When** figures are checked by hand before a cycle closes
- **Then** the check, the person and the outcome are recorded
- **Never** relying on someone remembering to look
- **Not proven yet** — specified, no test behind it

`R15.14` A channel-specific SKU code never becomes the master code

- **When** a channel uses its own identifier
- **Then** it is stored as a mapping against our SKU
- **Never** adopting the channel’s code as the item code, which breaks the moment a second channel does the same
- **Not proven yet** — specified, no test behind it

`R15.15` A size recommendation is advice, never a silent substitution

- **When** a fit suggestion is offered
- **Then** it is shown as a recommendation the customer chooses
- **Never** changing the size on an order on the customer’s behalf
- **Not proven yet** — specified, no test behind it

`R15.16` An order held past its cut-off is escalated, not queued

- **When** an order approaches the channel’s dispatch deadline
- **Then** it is raised to the person who can act, naming the deadline
- **Never** letting it age quietly into a penalty
- **Not proven yet** — specified, no test behind it

`R15.17` Closing stock is opening plus in minus out

- **When** a stock position is computed for a period
- **Then** closing = opening + receipts − issues, from the movements themselves
- **Never** carrying a maintained closing figure that can drift from the movements that produced it
- **Proven** by `core/tests/core.test.js › a receipt then an issue leaves the right number`

`R15.18` Courier return, customer return and wrong return cost three different things

- **When** a return is processed
- **Then** a courier return costs repacking only, a customer return costs alteration plus iron plus packing at the rate set for that design, and a wrong return is written off at the full selling price
- **Never** applying one blended return cost to all three, which hides the expensive kind inside the cheap kind
- **Not proven yet** — specified, no test behind it

`R15.19` A wrong return is never added back to stock

- **When** a return is found to be a different item from the one sent
- **Then** it becomes dead stock and the selling price is recognised as a loss
- **Never** restocking it, at any value, however sellable it looks
- **Proven** by `brand/suite/studio/verify_studio.js › sale minus return is the net, and net plus wrong return is the inventory`

Module 16 · HR & Payroll

Pay everyone right, on time

Office staff on a monthly salary and karigars paid by the piece, in one register, with attendance driving both and the festival advance already deducted.

Reads from	Manufacturing
Writes to	Accounting & GST
Apps	5
Rules	22, of which 8 are proven by a test that runs

The 5 apps in this module

Staff & Contractors — SPECIFIED — designed, not built

Attendance marked by tap, effective-dated salary, and karigar piece-rate earnings in a single register.

Time-off & Advances — SPECIFIED — designed, not built

Leave, Diwali advances, and exactly how they change this month’s payout before you approve it.

Appraisal & Hiring — SPECIFIED — designed, not built

Performance reviews and a hiring pipeline that ends in an employee record.

Recruitment — SPECIFIED — designed, not built

The pipeline before someone becomes an employee — an opening, the people who applied for it, a trial piece where the work itself is the interview, and the decision with its reason kept. It matters more here than in most trades: a karigar is taken on for skill on a particular garment, and the trial output is the evidence, so it is recorded against the design and the rate that would apply rather than remembered as an impression. A candidate who is not taken on now stays findable when the same skill is needed in a busy month, and their personal documents are held under the same consent and retention rules as anyone else's, not in a folder on somebody's phone.

Payout Execution — SPECIFIED — designed, not built

Where the calculation in the earnings register actually turns into money leaving the business — bank batch, UPI, cash against a signed receipt — with the method and the reference recorded against every payout, so the register's total and the money that actually moved can always be checked against each other.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

This month's register · staff and karigars

- **Figures across the top** — On roll 86 · Present today 79 · Advances out ₹2.4 L · Payout due ₹28.6 L
- **Columns** — Person · Basis · Days / pieces · Earned · State
- **Rows** — 4 worked examples, the first reading: A. Deshpande · Monthly · 26 · ₹64,000 · Approved
- **Controls** — Attendance marked on time · Payouts released on the 7th

The 22 rules this module must satisfy

`R16.1` A raise closes the old row, it does not overwrite it

- **When** a salary or rate changes
- **Then** the row in force is closed on the day before, and a new row opens
- **Never** editing the existing figure, which rewrites what the person was actually paid last year
- **Proven** by `core/tests/core.test.js › a raise closes the open row instead of overwriting it`

`R16.2` History resolves to what was actually in force

- **When** a past month is recomputed
- **Then** the rate in force in that month is used
- **Never** recomputing an old payslip at today's rate
- **Proven** by `core/tests/core.test.js › history still resolves to what was actually in force`

`R16.3` A future-dated raise activates by itself

- **When** a raise is entered with a future date

- **Then** it takes effect when that month arrives, with nobody remembering to apply it
- **Never** requiring a manual step, which is how an agreed raise is missed
- **Proven** by `core/tests/core.test.js › a future-dated raise activates by itself when that month arrives`

`R16.4` A month with nothing in force raises, and never returns zero

- **When** no rate covers the month being computed
- **Then** the computation is refused and the gap is named
- **Never** returning zero, which pays a real person nothing and looks like a valid answer
- **Proven** by `core/tests/core.test.js › a nothing-in-force month raises, and never returns zero`

`R16.5` Backdating over an open row is refused

- **When** a change is entered with a date inside a period already settled
- **Then** it is refused
- **Never** silently rewriting history that has already been paid and posted
- **Proven** by `core/tests/core.test.js › backdating over an open row is refused — that would rewrite history`

`R16.6` A person can leave and come back

- **When** someone rejoins after a break
- **Then** the spell log holds both periods and the gap between them
- **Never** creating a second employee record, which splits their history and their service
- **Proven** by `core/tests/core.test.js › a spell log lets a person leave and come back`

`R16.7` Month spans handle February and the year end

- **When** a period is computed across month or year boundaries
- **Then** the real calendar is used
- **Never** assuming thirty-day months, which is wrong twelve times a year and badly wrong in February
- **Proven** by `core/tests/core.test.js › month spans handle February and the year end`

`R16.8` Staff and piece-rate workers sit in one register

- **When** payroll is prepared
- **Then** monthly staff and piece-rate workers are computed in the same run and paid from the same register
- **Never** running two payrolls that have to be added together by hand
- **Not proven yet** — specified, no test behind it

`R16.9` An advance is recovered against a named advance

- **When** a deduction is made at payout
- **Then** it names the advance it is recovering and reduces that balance
- **Never** deducting an amount that cannot be traced to a specific advance

- **Not proven yet** — specified, no test behind it

`R16.10` Attendance drives pay, and both are visible together

- **When** a payout is computed
- **Then** the attendance it was computed from is shown beside it
- **Never** presenting a pay figure whose basis the person being paid cannot see
- **Not proven yet** — specified, no test behind it

`R16.11` Identity documents are read, never stored in a file that leaves

- **When** Aadhaar, PAN, bank or UPI detail is used for a computation
- **Then** it is used and not serialised into any exported or committed artifact
- **Never** writing personal identifiers into a report, a backup file or a repository
- **Not proven yet** — specified, no test behind it

`R16.12` A payout that fails to post does not mark as paid

- **When** the bank transfer or the ledger posting fails
- **Then** the payout stays unpaid and the failure is raised
- **Never** marking it paid on submission, which loses a real person’s wages in the gap
- **Not proven yet** — specified, no test behind it

`R16.13` The daily rate is the monthly salary divided by twenty-seven

- **When** a day of attendance is priced
- **Then** the daily rate is that month’s salary ÷ 27, using the salary in force in that month
- **Never** using calendar days, working days, or a rate carried over from a month with a different salary
- **Not proven yet** — specified, no test behind it

`R16.14` Attendance codes have fixed multipliers and a blank is absent

- **When** earned pay is computed from attendance
- **Then** present, holiday, on-duty and paid leave count 1, a half day counts 0.5, absent and unpaid leave count 0, and an empty cell counts as absent
- **Never** treating a blank as present, or as unknown to be filled in later — a blank that pays is a blank that will be left blank
- **Not proven yet** — specified, no test behind it

`R16.15` Threshold hours do not move when salary moves

- **When** a raise takes effect
- **Then** the monthly hour threshold for that role stays as it was
- **Never** scaling the threshold with the salary, which silently changes what the person is expected to work in exchange for a raise
- **Not proven yet** — specified, no test behind it

`R16.16` Productivity cost is that month’s salary over the threshold, times hours worked

- **When** the cost of a person’s time is charged to work
- **Then** it is (salary in force that month ÷ threshold hours) × the hours actually active
- **Never** using a single annual figure, which misprices every month on either side of a raise
- **Not proven yet** — specified, no test behind it

`R16.17` A holiday is paid and produces no hours

- **When** a holiday is marked
- **Then** it pays a full day and contributes zero productive hours
- **Never** counting holiday hours as production, which flatters every efficiency figure that reads them
- **Not proven yet** — specified, no test behind it

`R16.18` A half day is half the hours, from the same start

- **When** a half day is marked
- **Then** it starts at the normal in-time and its hours are half the full shift for that person’s pattern
- **Never** assuming a fixed midday finish for everyone, when the male and female shift lengths differ
- **Not proven yet** — specified, no test behind it

`R16.19` The festival flag drives leave and nothing else

- **When** a religion is recorded against a person
- **Then** it is used only to match a festival-leave request
- **Never** using it as a filter, a grouping or a report dimension anywhere else in the system
- **Not proven yet** — specified, no test behind it

`R16.20` A geofence failure flags, it does not refuse

- **When** attendance is marked outside the radius set for the unit, or outside the grace window
- **Then** it is recorded with the flag and raised to the manager
- **Never** refusing the mark — a system that locks someone out of being paid for standing at the wrong gate has failed at its actual job
- **Not proven yet** — specified, no test behind it

`R16.22` A shared document carries the pay rules, never the pay roster

- **When** a plan, a specification or any document that leaves this building is generated
- **Then** it carries the formulas, thresholds and effective-dating that decide pay, and refers to the roster rather than reproducing it
- **Never** printing an individual’s name beside their salary, or their religion at all, into a document that is committed to a repository and travels with every copy — the software needs those fields, a reader of the plan does not
- **Not proven yet** — specified, no test behind it

`R16.21` An override is allowed and is always recorded

- **When** an administrator corrects attendance, a geofence flag or a payroll figure
- **Then** the change, the person and the reason go to the audit trail
- **Never** an override that leaves no trace, which is indistinguishable from the system having been wrong
- **Proven** by `core/tests/core.test.js › an update records what it was as well as what it became`

Module 17 · Marketing

Sell more without cutting the price

Plan the festive calendar, run the campaigns, and let rules keep you competitive on the panels without giving the margin away.

Reads from	Inventory & Catalog, CRM
Writes to	Sales, E-commerce / OMS
Apps	8
Rules	10, of which 0 are proven by a test that runs

The 8 apps in this module

Social Calendar — SPECIFIED — designed, not built

Plan and publish across every channel from one calendar.

Campaigns — SPECIFIED — designed, not built

Email, SMS and WhatsApp campaigns measured on real revenue, not opens.

Repricing Engine — SPECIFIED — designed, not built

Rules per panel and per design — floor, ceiling, match-lowest and a festive override — so a Diwali sale does not quietly go below cost. And what each change actually did: a design whose orders fell after a price rise shows as exactly that, next to the rule that raised it.

Automation — SPECIFIED — designed, not built

If this happens, do that — across any module, without writing code.

Blog & Pages — SPECIFIED — designed, not built

How to drape it, what to wear it to, which fabric for which season — written, scheduled and published to your own site with the meta and internal links already set.

Events — SPECIFIED — designed, not built

Trade shows and exhibitions worked as a channel of their own — booth, budget and every lead captured on the floor landing straight in CRM instead of on a stack of business cards.

Website & Page Builder — SPECIFIED — designed, not built

The storefront itself, built by dragging sections into place rather than by editing a theme file — hero, product grid, size guide, lookbook, contact form — each block reading live from the catalogue, so a price or a stock state on a landing page is the same number the order screen uses instead of a figure someone pasted in and forgot. Blog & Pages above writes articles into a site that already exists; this is for the businesses that do not have one, and it is the gap that shows up plainly when this module list is set beside a mature open-source ERP: they ship a full site builder next to the blog, and until now this did not.

Markdown / Clearance Optimization — SPECIFIED — designed, not built

The same rule engine that reprices for competitiveness, aimed at ageing stock instead: when to start discounting it and by how much, before it becomes a warehouse write-off rather than a sale at a lower margin.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Campaigns measured on revenue, not opens

- **Figures across the top** — Spend, month ₹4.8 L · Revenue from it ₹31.2 L · ROAS 6.5× · Repricing rules 24
- **Columns** — Campaign · Channel · Spend · Revenue · ROAS
- **Rows** — 4 worked examples, the first reading: Navratri launch · Email · ₹42,000 · ₹6,10,000 · 14.5×
- **Controls** — Priced above floor everywhere · Posts published on schedule

The 10 rules this module must satisfy

`R17.1` A campaign is measured on revenue, not on opens

- **When** campaign performance is reported
- **Then** it is attributed to actual orders
- **Never** reporting opens and clicks as the result, which measures the message rather than the business
- **Not proven yet** — specified, no test behind it

`R17.2` A repricing rule shows what it did

- **When** a rule changes a price
- **Then** the change, the rule that made it and the effect on orders are recorded together
- **Never** changing prices with no record, which makes a bad rule impossible to identify or reverse
- **Not proven yet** — specified, no test behind it

`R17.3` A price floor is a floor

- **When** a repricing rule would go below the floor set for a SKU
- **Then** it stops at the floor
- **Never** undercutting to match a competitor below cost
- **Not proven yet** — specified, no test behind it

`R17.4` A markdown starts before the stock is dead, not after

- **When** stock reaches the age set for it
- **Then** the markdown schedule begins
- **Never** waiting until it is unsellable, which converts a lower-margin sale into a write-off
- **Not proven yet** — specified, no test behind it

`R17.5` A campaign cannot message someone who has not consented

- **When** a marketing send is prepared
- **Then** the recipient list is filtered by consent at send time
- **Never** sending to a list captured before the consent was checked
- **Not proven yet** — specified, no test behind it

`R17.6` A published page reads live catalogue data

- **When** a page shows a price or a stock state
- **Then** it reads the same record the order screen reads
- **Never** pasting a figure into the page, which goes stale the first time the price changes
- **Not proven yet** — specified, no test behind it

`R17.7` An exhibition is a channel

- **When** leads and sales come from a trade show
- **Then** they land in CRM and the order book against that channel
- **Never** collecting them on paper to be entered later, which is where they are lost
- **Not proven yet** — specified, no test behind it

`R17.8` A marketing automation cannot move money

- **When** a campaign rule fires
- **Then** it may message, tag, schedule or reprice within its limits
- **Never** issuing a refund, a credit note or a payment — that is not what this engine is allowed to do
- **Not proven yet** — specified, no test behind it

`R17.9` A scheduled post that fails is reported as failed

- **When** a scheduled publication does not go out
- **Then** it is raised with the reason
- **Never** showing it as published in the calendar while nothing was posted
- **Not proven yet** — specified, no test behind it

`R17.10` Return on ad spend is measured against real orders

- **When** campaign performance is computed
- **Then** it is revenue from attributed orders ÷ spend actually incurred

- **Never** using a platform's own reported conversions as the revenue figure, which counts orders this system has no record of
- **Not proven yet** — specified, no test behind it

Module 18 · AI Content Engine

Write it, shoot it, cut it — from your own catalogue

Listings, ads, reels and product photography generated from your own designs, in a voice that sounds like one person from Surat rather than a template — so the words match the piece and the picture is the size Myntra actually wants.

Reads from	Inventory & Catalog
Writes to	Marketing, E-commerce / OMS
Apps	8
Rules	11, of which 2 are proven by a test that runs

The 8 apps in this module

Content Engine — SPECIFIED — designed, not built

Fourteen stages in your own voice — buyer psychology, competitor reading, hooks, the product description, marketplace copy for Amazon and Myntra, ad variations, reel scripts, song lyrics for the reel, the calendar, size chart and alt text.

Image Studio — SPECIFIED — designed, not built

A phone photo becomes a listing image: layers, free transform, background removal, Myntra 1080×1440 and every other channel preset, watermark and SEO alt text.

Video Studio — SPECIFIED — designed, not built

Text and image to video, reels and ad cuts sized for every channel.

Design Studio — SPECIFIED — designed, not built

Banners, festive creatives and thumbnails — templates, layers, undo and redo, any colour, exact sizing and stock elements, exporting PNG, JPG or PDF at whatever size the panel or the printer asks for.

Motion Renderer — **ENGINE** — the arithmetic runs on the command line, no screen yet

A reel rendered from a page of HTML and CSS — the same layers, fonts and brand colours the Design Studio already uses — into a real MP4, on this machine, with nothing uploaded. It is not a screen recording: the clock is faked, the animation is seeked to the exact instant of each frame, and only then is that frame captured, so the render does not care whether the machine was busy. Rendering the same festival banner twice produces the

same file to the byte, which is what makes a reel something you can check and re-cut rather than something you have to watch all the way through and hope about.

Narration Studio — SPECIFIED — designed, not built

A voice over the reel, in the language the buyer actually speaks — the same script the Content Engine wrote, spoken. The default needs nothing installed and no key, because every modern browser can already speak; a self-hosted or cloud voice sits behind it as an interchangeable provider for anyone who wants a cloned or branded one. Long scripts are split at sentence boundaries and rejoined, so a two-minute description is not cut off at whatever limit a service imposes. A voice cloned from a real person is only ever used with that person's recorded consent, filed against them in Data Privacy & Consent like any other permission.

Image Generation Slot — SPECIFIED — designed, not built

Generated imagery — a model on a background you do not have to shoot, a festival backdrop, a lifestyle scene — as a capability with interchangeable providers rather than a bet on one service: a queue of jobs, a preview while it works, inpainting to fix one region, upscaling and face correction. This one needs a graphics card. Image models cannot run on an ordinary office machine or on the container this system is built in, so what ships is the queue, the review screen and the provider slot, with the generating itself done by whichever engine you point it at — your own GPU box, or a cloud service, swapped without touching anything else. Said plainly here because the alternative is a screen that looks finished and produces nothing.

Publisher — SPECIFIED — designed, not built

One push sends the listing, images and copy to the website and every panel, and reports back what went live and what a panel rejected, with the reason.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Content pipeline · written from your own designs

- **Figures across the top** — Listings this week 184 · Images produced 612 · Reels cut 38 · Rejected by panel 2
- **Columns** — Design · What was made · Channel · State
- **Rows** — 4 worked examples, the first reading: VG-1180 · Listing · title, bullets, A+ · Amazon · Live
- **Controls** — Written from real catalogue data · Published in one push

The 11 rules this module must satisfy

`R18.1` Content is written from the catalogue, not about the category

- **When** a listing or description is generated
- **Then** it is generated from that product's own attributes
- **Never** writing plausible copy about the kind of thing it is, which is how a listing describes features the product does not have
- **Not proven yet** — specified, no test behind it

`R18.2` Structured fields get keywords; anything a human reads gets feeling

- **When** text is produced for a back-end field versus a caption
- **Then** each is written for its own reader
- **Never** writing both the same way, which is the clearest signal of machine-written content
- **Not proven yet** — specified, no test behind it

`R18.3` Product nouns are banned from creative surfaces

- **When** a caption or a hook is written
- **Then** the product noun is excluded
- **Never** letting search vocabulary bleed into copy meant to be felt
- **Not proven yet** — specified, no test behind it

`R18.4` The engine criticises its own draft before anyone sees it

- **When** a draft is produced
- **Then** it is put through the self-critique pass and the second draft is what is shown
- **Never** showing the first attempt, which is rarely the best one
- **Not proven yet** — specified, no test behind it

`R18.5` A render is seeked, never recorded

- **When** a video is produced from a page
- **Then** the clock is driven by hand and each frame is captured at its exact instant
- **Never** playing the animation and recording the screen, which bakes whatever else the machine was doing into the customer's reel
- **Proven** by `brand/suite/studio/motion_render.js › a second render produces frame-for-frame identical images`

`R18.6` The same scene renders to the same file

- **When** a render is repeated
- **Then** the output is identical to the byte
- **Never** producing a different file each time, which makes the output impossible to check or approve
- **Proven** by `brand/suite/studio/motion_render.js › and a byte-identical MP4`

`R18.7` A generated asset is labelled as generated

- **When** an image or video is produced by a model
- **Then** it carries that fact in the asset record
- **Never** letting a generated image become indistinguishable from a photograph of the actual product
- **Not proven yet** — specified, no test behind it

`R18.8` Generation stays badged a mockup until a real provider is wired

- **When** a capability is demonstrated without a live provider behind it
- **Then** it is labelled a mockup wherever it appears

- **Never** showing a simulated render as a finished one
- **Not proven yet** — specified, no test behind it

`R18.9` Image generation states that it needs a graphics card

- **When** the image generation slot is opened with no provider attached
- **Then** it says so plainly and produces nothing
- **Never** presenting a finished-looking screen that cannot generate anything
- **Not proven yet** — specified, no test behind it

`R18.10` A cloned voice needs the consent of the person it came from

- **When** a voice is cloned for narration
- **Then** that person's recorded consent is on file against them
- **Never** cloning from a recording merely because it was available
- **Not proven yet** — specified, no test behind it

`R18.11` A publish reports what actually went live

- **When** content is pushed to several destinations
- **Then** each result comes back individually, with reasons for rejections
- **Never** reporting one overall success, which leaves the business absent where it believes it is present
- **Not proven yet** — specified, no test behind it

Module 19 · SEO, AEO & AIO

Be found by a search box, an answer box and an AI

Content already exists once this module is reached; here it is made findable — by a traditional search engine, by the answer box above the results, and by the AI assistants now answering shopping questions directly instead of sending someone to a results page.

Reads from	Inventory & Catalog, AI Content Engine
Writes to	Marketing
Apps	3
Rules	6, of which 0 are proven by a test that runs

The 3 apps in this module

Technical SEO & Schema — SPECIFIED — designed, not built

Structured data, sitemaps and page-level technical checks against your own storefront and content pages, so a search engine can read what a page is actually about instead of guessing from the text alone.

Answer-Engine Optimization — SPECIFIED — designed, not built

Content shaped to be quoted directly by an answer box or a voice assistant — a clear, citable answer near the top of the page — rather than written only for a person to scroll through.

AI-Engine Visibility Tracking — SPECIFIED — designed, not built

Whether and how this business is actually cited when someone asks an AI assistant a shopping question in this category, tracked over time — the same discipline as rank tracking, aimed at a newer kind of result page.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Found by a search box, an answer box and an assistant

- **Figures across the top** — Tracked queries 214 · On page one 68 · Cited by an assistant 19 · Listings with schema 97%
- **Columns** — Query · Position · Assistant cites us · Page
- **Rows** — 4 worked examples, the first reading: anarkali suit for wedding · 4 · Yes · /anarkali
- **Controls** — Pages whose markup matches what is on them · Claims marked up that the page does not make

The 6 rules this module must satisfy

`R19.1` Structured data describes what is actually on the page

- **When** schema markup is generated
- **Then** it is generated from the same record the page renders
- **Never** marking up a price or availability that differs from the page, which is penalised and deserved
- **Not proven yet** — specified, no test behind it

`R19.2` A ranking figure names where it was measured

- **When** a position or citation is reported
- **Then** the engine, the query and the date are stored with it
- **Never** reporting a bare position, which cannot be compared with anything
- **Not proven yet** — specified, no test behind it

`R19.3` An answer-shaped page still says the same thing as the product record

- **When** content is shaped to be quoted by an answer box
- **Then** the claims match the catalogue
- **Never** writing a more quotable claim than the product supports
- **Not proven yet** — specified, no test behind it

`R19.4` A technical fix is verified on the live page

- **When** a technical SEO issue is marked resolved
- **Then** the live page is re-fetched and re-checked
- **Never** closing it because the change was deployed
- **Not proven yet** — specified, no test behind it

`R19.5` AI-engine visibility is tracked over time, not sampled once

- **When** citation in an AI answer is measured
- **Then** it is measured repeatedly and stored as a series
- **Never** quoting a single lucky result as the position
- **Not proven yet** — specified, no test behind it

`R19.6` A sitemap lists only pages that exist and are meant to be found

- **When** a sitemap is generated
- **Then** it contains live, indexable pages
- **Never** listing archived or blocked pages, which wastes the crawl on nothing
- **Not proven yet** — specified, no test behind it

Module 20 · Projects & Collaboration

The work that is not an order — and the talking around it

An exhibition in Hyderabad, a boutique's custom order, a new godown fit-out, a legal matter with a supplier. Work that is not a sales order still has a deadline, a cost and documents — and it belongs on the same records as everything else.

Reads from	CRM, Sales, HR & Payroll, Inventory & Catalog
Writes to	Accounting & GST, HR & Payroll, CRM
Apps	7
Rules	9, of which 1 are proven by a test that runs

The 7 apps in this module

Projects & Cases — SPECIFIED — designed, not built

An exhibition, a custom order for a chain, a fit-out or a dispute — stages you define, owners, deadlines, documents, hours and real cost, all on one record the ledger can see.

Timesheets & Planning — SPECIFIED — designed, not built

Who is on what this week and the hours that actually went in — against a project, an exhibition or a machine — with billable and non-billable kept apart.

Approvals — SPECIFIED — designed, not built

One queue for everything waiting on a yes: a mill purchase order, a boutique discount, a leave day, a credit note, a payment. The rule that sent it there is next to it, and the decision goes on the record.

Forum — SPECIFIED — designed, not built

Questions and answers that outlive a chat — for customers, dealers or staff — with the useful ones kept where the next person will actually find them.

Automation Studio — SPECIFIED — designed, not built

The place a person builds “when this happens, do that” by dragging it out and watching it run — a trigger, the steps after it, a branch where the answer decides which way to go — over the same event stream every module already writes to. Marketing’s Automation aims that idea at campaigns; this is the general one, reaching any module: a payment marked short holds the next dispatch and opens a claim, a karigar’s pooled sets crossing a threshold raises the payout for approval, a return marked damaged writes off the piece and messages the buyer. Every run is kept — what fired it, each step, what each step returned — because an automation nobody can inspect afterwards is a rule the business cannot trust with its money.

Discuss — SPECIFIED — designed, not built

The conversation attached to the record it is about — this order, this mill bill, this dispute — so a year later the reason for the decision is still sitting beside it.

Knowledge Base — SPECIFIED — designed, not built

A searchable internal wiki of standard operating procedures, scoped to the role it applies to, so how a task is meant to be done is written down once instead of carried in one person’s head.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Live work · exhibitions, custom orders, disputes

- **Figures across the top** — Open items 34 · Hours this month 1,248 · Unbilled cost ₹18.4 L · Due this week 6
- **Columns** — Item · Party · Stage · Hours · Due
- **Rows** — 4 worked examples, the first reading: EXH-HYD-26 · Hyderabad exhibition · Stall & stock ready · 62 · 4 Aug
- **Controls** — Hours captured same day · Costs posted within 7 days

The 9 rules this module must satisfy

`R20.1` Billable time becomes an invoice line without retyping

- **When** approved time exists against a project
- **Then** the rate card turns it into an invoice line and a real cost

- **Never** re-entering hours into an invoice, which is where the two figures start to differ
- **Not proven yet** — specified, no test behind it

`R20.2` Billable and non-billable are separated at entry

- **When** time is recorded
- **Then** it is marked billable or not as it is entered
- **Never** deciding at invoice time, which quietly turns unbillable work into a charge
- **Not proven yet** — specified, no test behind it

`R20.3` An approval shows the rule that demanded it

- **When** anything lands in the approvals queue
- **Then** the rule that sent it there is displayed beside it
- **Never** presenting a request with no stated reason, which makes approval a formality
- **Not proven yet** — specified, no test behind it

`R20.4` An approval decision goes to the audit trail

- **When** a request is approved or refused
- **Then** the decision, the person and the time are recorded
- **Never** recording only the outcome on the record, which loses who accepted the risk
- **Proven** by `core/tests/core.test.js › an update records what it was as well as what it became`

`R20.5` An automation run is kept step by step

- **When** a rule fires
- **Then** what triggered it, each step, and what each step returned are stored
- **Never** keeping only the outcome — an automation nobody can inspect afterwards is a rule the business cannot trust with its money
- **Not proven yet** — specified, no test behind it

`R20.6` An automation acts within a named scope

- **When** a rule is built
- **Then** the records it may read and write are declared on it
- **Never** letting a rule reach anywhere in the system because it happens to run as an administrator
- **Not proven yet** — specified, no test behind it

`R20.7` A project cost includes the time and the material

- **When** project profitability is computed
- **Then** labour, material and expenses booked to it are all included
- **Never** reporting on revenue and time alone, which shows a loss-making project as profitable
- **Not proven yet** — specified, no test behind it

`R20.8` A decision is recorded where the decision was made

- **When** a discussion resolves something
- **Then** it is attached to the record it concerns
- **Never** leaving the reasoning in a chat thread that will not be found in a year
- **Not proven yet** — specified, no test behind it

`R20.9` A procedure is scoped to the role it applies to

- **When** a standard procedure is published
- **Then** it is scoped to the role that performs it
- **Never** publishing one undifferentiated manual that nobody reads
- **Not proven yet** — specified, no test behind it

Module 21 · Dashboard & BI

See the whole house without asking anyone

Every number rolls up here as work happens — the day's marketplace orders, what the karigars finished, what is still lying at the dyer, what the mills are owed. No exports, no waiting for month-end.

Reads from	Every module
Writes to	—
Apps	5
Rules	9, of which 6 are proven by a test that runs

The 5 apps in this module

CEO Dashboard — **BROWSER APP** — opens and self-tests, no shared database behind it

Cash, sales by channel, stock by design, profit per piece and the alerts that matter — one screen, refreshed as the day runs.

Report Builder — **BROWSER APP** — opens and self-tests, no shared database behind it

Drag the fields you want into a report and save it for the whole team.

Group Consolidation — **BROWSER APP** — opens and self-tests, no shared database behind it

Ethnic Fashion, Vastrangam and Adini Couture as one set of figures, inter-company transfers removed, so the group position is real rather than three spreadsheets added together. Adini Couture has no registration of its own and mainly does job work — it still counts in the group, without being pulled into a return it does not belong in. Add the fourth company the day you open it.

Excel Dashboard Builder — SPECIFIED — designed, not built

A full workbook — financial summary, HR, purchase, sales, inventory and production, GST, expenses — generated from the live records behind every other screen, with each company shown as its own row and a consolidated row that is a formula over them, never a separately typed total.

ESG / Sustainability Reporting — SPECIFIED — designed, not built

Water usage, chemical compliance, waste and packaging, reported from the same certificate and audit records Quality & Compliance already keeps — so a sustainability report is a query over evidence already on file, not a separate exercise assembled once a year from scratch.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Group dashboard · Vastrangam + ethnic arm · FY 2026-27

- **Figures across the top** — Revenue, this month ₹1.42 Cr · Cash + bank ₹38.4 L · Stock at cost ₹2.09 Cr · Overdue from boutiques ₹6.2 L
- **Columns** — Channel · Orders · Net revenue · vs last month
- **Rows** — 4 worked examples, the first reading: Marketplaces · 3,092 · ₹68.3 L · +7%
- **Controls** — Anarkali & gown sets · Sarees · Kurta sets

The 9 rules this module must satisfy

`R21.1` The group figure is the sum minus inter-company trade

- **When** several companies are consolidated
- **Then** entries naming a counterparty inside the group are eliminated, and gross, eliminated and group are all shown
- **Never** presenting the plain sum as the group result, which inflates turnover by trade the group never did with the outside world
- **Proven** by `core/tests/core.test.js › the group is the sum MINUS what the companies sold each other`

`R21.2` An entry cannot be its own counterparty

- **When** an entry names a counterparty company
- **Then** it is refused if that is the same company
- **Never** allowing a company to trade with itself, which eliminates a figure that was never doubled
- **Proven** by `core/tests/core.test.js › an entry cannot be its own counterparty`

`R21.3` The number of companies is data, not a constant

- **When** the group grows
- **Then** a company is a row and every consolidation follows
- **Never** building around a fixed number of companies or channels

- **Proven** by `core/tests/core.test.js › ten companies and ten channels each is a hundred channels, not a limit`

`R21.4` Every dashboard figure is a live query

- **When** a KPI is displayed
- **Then** it is computed from the ledger and the stock table at that moment
- **Never** reading a maintained summary table, which can be wrong without looking wrong
- **Proven** by `core/tests/core.test.js › the trial balance is computed from the lines, never stored`

`R21.5` A consolidated row is a formula over the company rows

- **When** a workbook or report shows a consolidated figure
- **Then** it is computed from the company rows beside it
- **Never** typing a separate consolidated total, which is a second copy that will disagree
- **Proven** by `brand/suite/studio/verify_studio.js › the totals row is the sum of the rows above it`

`R21.6` A figure a user may not see is not returned

- **When** a report runs for a scoped user
- **Then** out-of-scope rows are excluded from the query
- **Never** computing the full figure and hiding part of it in the display
- **Proven** by `core/tests/core.test.js › one company cannot read another company`

`R21.7` An exported report says when it was taken

- **When** a report is exported
- **Then** the as-at time and the filters are printed on it
- **Never** producing an undated export, which is quoted months later as though it were current
- **Not proven yet** — specified, no test behind it

`R21.8` A saved report keeps its definition, not its results

- **When** a report is saved and re-run
- **Then** the definition re-runs against current data
- **Never** storing a snapshot and presenting it as live
- **Not proven yet** — specified, no test behind it

`R21.9` A figure with no drill-down is a defect

- **When** any total is shown
- **Then** it opens to the records beneath it
- **Never** shipping a number that cannot be explained by clicking it
- **Not proven yet** — specified, no test behind it

Module 22 · AI Assistant, Agents & Automation

Ask the house a question — and let the routine work run itself

Last for the same reason Dashboard & BI is late: something that answers questions about the whole business can only be built once the whole business is in one place. Three different things live here and the difference between them matters. An ASSISTANT answers a question you asked, from the records, with the records attached. A CHATBOT holds the same conversation with your customer instead of you. An AGENT is given a job rather than a question and works out the steps itself. That last one is what separates this module from Automation Studio in Module 20, where a person draws the steps in advance and the rule runs the same way every time; and from Automation in Module 17, which fires marketing campaigns and nothing else. Both of those stay exactly as they are — this module sits above them and calls them, rather than replacing either. Module 18 writes content; this module answers and acts.

Reads from	Every module
Writes to	Projects & Collaboration, CRM, Marketing
Apps	5
Rules	15, of which 2 are proven by a test that runs

The 5 apps in this module

AI Assistant — SPECIFIED — designed, not built

Ask in your own words — “what did Myntra actually pay us last week, and what is still short?”, “which designs did Kalamandir return most” — and get the answer with the rows it came from underneath it, each clicking through to the record. It reads the same ledger and the same settlement lines the accounts screen reads, so its figure and the books agree. When it cannot find the answer it says so; it never puts a plausible number in place of a real one.

AI Chatbot — SPECIFIED — designed, not built

The same engine facing your buyer, on the website and on WhatsApp: where is my order, will the L fit me, I want to return this saree. It reads the real order and the real size chart, not a script written last season, and it says “let me get someone” for anything about money or a complaint — handing over into the Surat helpdesk queue with the whole chat already attached. It never asks a buyer for a card number, a bank detail or a password.

AI Agents — SPECIFIED — designed, not built

A job rather than a question: “chase every unmatched payout line from last week and draft the claim for each.” It works out the steps and stops where a person has to decide. Filing the claim, moving money, changing a price or messaging a boutique all wait for your yes.

Agent Guardrails & Run Log — SPECIFIED — designed, not built

What each agent is allowed to touch, written down as a scope rather than trusted to a prompt, and every run recorded step by step: what started it, what it read, what it proposed, what a person approved, what it actually

changed. Kept in the same audit trail as everything else, with the same absence of an off switch. An agent whose working nobody can inspect afterwards is not a colleague, it is an unexplained entry in the books.

Knowledge & Retrieval — SPECIFIED — designed, not built

The index that makes the answers grounded: your own designs, rate cards, settlement files, standard procedures and past decisions, searchable so a reply quotes what is actually on file instead of what a model remembers about the trade in general. Permission-scoped at the row, so two people asking the same question get answers drawn only from what each of them may already see.

The 1 specified screen

Not a picture — the columns, the rows and the controls, written down so a built screen can be compared against something.

Agent runs · this week · every step recorded

- **Figures across the top** — Questions answered 412 · Answered from records 412 · Figures estimated 0 · Waiting on a person 7
- **Columns** — Run · What it was asked to do · Where it got to · State
- **Rows** — 5 worked examples, the first reading: AG-1187 · Chase 14 unmatched Myntra payout lines, draft a claim · 14 drafted, none filed · Needs a yes
- **Controls** — Answers carrying the records they came from · Runs a person can replay step by step · Money moved without a human yes

The 15 rules this module must satisfy

`R22.1` An answer carries the records it came from

- **When** the assistant answers a question about a figure
- **Then** the rows it used are attached and each one opens to its record
- **Never** giving a bare number, which cannot be checked and therefore cannot be trusted
- **Not proven yet** — specified, no test behind it

`R22.2` An unknown answer is said, never estimated

- **When** the assistant cannot find the figure
- **Then** it says so and shows what it looked at
- **Never** producing a plausible number — a confident wrong figure costs far more than an honest blank
- **Not proven yet** — specified, no test behind it

`R22.3` The assistant answers only from what the asker may already see

- **When** a question is asked by a scoped user
- **Then** retrieval is filtered to that user’s permissions before the answer is composed
- **Never** letting the assistant become a way around permissions that every other screen enforces
- **Not proven yet** — specified, no test behind it

`R22.4` An agent cannot widen its own scope

- **When** an agent runs
- **Then** it works within the records and the spend it was given
- **Never** expanding its scope mid-run, however sensible the next step would be
- **Not proven yet** — specified, no test behind it

`R22.5` Money never moves without a human yes

- **When** an agent proposes a refund, a payment, a payout or a credit note
- **Then** it stops and waits for a person
- **Never** executing it, no matter how confident or how small the amount
- **Not proven yet** — specified, no test behind it

`R22.6` A customer is never messaged by an agent without approval

- **When** an agent drafts a message to a real customer
- **Then** a person approves it before it is sent
- **Never** sending on the agent's own judgement
- **Not proven yet** — specified, no test behind it

`R22.7` A price is never changed by an agent alone

- **When** an agent proposes a price change
- **Then** it enters the approvals queue with the reasoning attached
- **Never** writing the new price directly
- **Not proven yet** — specified, no test behind it

`R22.8` Every agent run is replayable step by step

- **When** an agent finishes, stops or fails
- **Then** what started it, what it read, what it proposed and what was approved are all recorded
- **Never** keeping only the outcome — an unexplained change made by software is worse than one made by a person
- **Not proven yet** — specified, no test behind it

`R22.9` Agent spending goes through the same ceiling as everything else

- **When** an agent calls a paid provider
- **Then** it is routed through the Provider Router and refused past the ceiling
- **Never** giving an agent its own unmetered budget
- **Proven** by `brand/suite/router.js › the third call would break the ceiling and is refused`

`R22.10` The chatbot hands over rather than guessing about money

- **When** a customer asks about a refund, a charge or a complaint
- **Then** it hands to a person with the whole conversation attached

- **Never** answering from a general idea of the policy
- **Not proven yet** — specified, no test behind it

`R22.11` The chatbot never asks a customer for a credential

- **When** a customer is identified in a chat
- **Then** identity is established through the order and the contact already on file
- **Never** asking for a card number, a bank detail or a password — the promise made everywhere else does not get a chatbot-shaped exception
- **Not proven yet** — specified, no test behind it

`R22.12` A handover lands in the existing queue

- **When** a conversation is passed to a person
- **Then** it enters the Module 04 Helpdesk queue with its history
- **Never** creating a second inbox that someone has to remember to watch
- **Not proven yet** — specified, no test behind it

`R22.13` An agent is not a hidden actor in the audit trail

- **When** an agent changes anything
- **Then** the change is attributed to the agent, its run, and the person who approved it
- **Never** recording it under a service account, which makes an automated change indistinguishable from a human one
- **Proven** by `core/tests/core.test.js › an audited insert leaves a before/after trail`

`R22.14` A retrieved document does not become an instruction

- **When** the assistant reads a document, a review or a message while answering
- **Then** that content is treated as data to report on
- **Never** following instructions found inside retrieved content, which is how a supplier’s PDF ends up steering the system
- **Not proven yet** — specified, no test behind it

`R22.15` An assistant answer is reproducible from the records it cites

- **When** the assistant states a figure
- **Then** re-running the same query over the same records gives the same figure
- **Never** an answer that cannot be reproduced, which is a guess with citations attached
- **Not proven yet** — specified, no test behind it

Part three — this business’s own engine

Everything above is the product: what any business running on it must do. This part is what THIS business does, and it is the half nobody else inherits. Every figure below is read out of the engine when this document is generated — none of it is typed here, so it cannot drift from the software that pays people.

The roster

22 people on file. The list below is who was on the floor on 2026-09-01 — recorded with the date it was true on, because a roster that means “now” quietly changes as time passes and somebody’s employment moves with it.

Person	Role	Employed	Pay basis
bharti	Dhaga Cutting	2025-04-01 → 2026-03-31	Attendance
esadul	Master	2025-04-01 → present	Attendance
ibrahim	Master	2025-08-01 → 2026-08-31	Attendance
ikram	Iron	2025-04-01 → present	Piece-rate
jamil	Master	2025-04-01 → gone, no date stated	Attendance
joginder	Iron	2025-04-01 → 2026-03-31	Hourly
kajal	—	2026-08-01 → present	Attendance
kalyani	—	2026-08-01 → present	Attendance
karim	Supervisor	2022-08-01 → present	Flat
krishna	Packing	2025-06-01 → gone, no date stated	Attendance
maasi	Dhaga Cutting	2025-04-01 → 2026-03-31	Attendance
muskan	Packing	2025-04-01 → present	Attendance
pooja	—	2026-08-01 → present	Piece-rate
priyanka	Dhaga Cutting	2026-04-01 → 2026-07-31	Attendance
rupsa	Dhaga Cutting	2026-04-01 → 2026-07-31	Attendance
sanjana	—	2026-08-01 → present	Attendance
sarfaraz	Master	2025-04-01 → gone, no date stated	Attendance
selima	Dhaga Cutting	2026-04-01 → 2026-07-31	Attendance
shivam	Packing	2025-06-01 → gone, no date stated	Attendance
surender	Iron	2025-06-01 → gone, no date stated	Attendance
trial_2026_08_a	—	no spell — a trial	—
upender	Iron	2026-04-01 → present	Flat

> **5 people are gone and no leaving date was ever stated.** *Their months from the snapshot on resolve as unresolved rather than as “not employed” — the two are different claims and only one of them is true. They pay nothing and stay on the report until a date is given.*

What a month pays

The owner, twice, in his own words: *"Salary calculation should be like Monthly Salary/monthly threshold hour"*. Both halves are read at the month being paid, because one person's salary and their threshold change on different dates and fixing either half to a year would be wrong for the months between them.

```
rate per hour = that month's salary ÷ that month's threshold hours
earned        = paid hours × rate per hour
```

Paid hours and productive hours are two different figures, and they part company on exactly two attendance codes. A holiday and a paid leave day carry a full day of pay and no productive time at all: the salaried are paid for them, the hourly and piece-rate never reach the attendance sheet, and the productive figure — the one that costs a design — still shows nothing was made.

Flat pay does not move with the month's length, so it carries a second number: what the hours would have earned, and the variance between that and the cash. Without it nothing makes short hours visible on a fixed salary.

What a piece of work pays

A piece rate belongs to an **operation on a garment**, not to a person. The owner states each one once and everybody doing that work is paid at it, which is why adding the fourth person to an operation is a row in the roster and not a rate somebody has to remember to copy.

Iron

Garment	Rate	In force from
Anarkali	7.5	2026-04-01
Pant/Plazo (bottom)	2.5	2026-04-01
Dupatta	2	2026-04-01
Top (Kurti/Kurta)	4	2026-04-01
Uniform Shirt	3.5	2026-04-01
Uniform Pant	2.5	2026-04-01

Dhaga Cutting

Garment	Rate	In force from
Anarkali/Kurti/Kurta	1.5	2025-04-01
Uniform Shirt/Pant	1.5	2025-04-01
Pant/Plazo/Bottom	1	2025-04-01
Dupatta	1	2025-04-01

A garment the card does not price is refused, not paid as zero. A garment two card entries both claim — the same word appearing in two slash-lists at different rates — is refused too: picking either would be a coin toss with somebody's wages on it.

Advances

The owner: *"Is advance amount, should not include in salary, keep it seperate, they will deduct later in few months, just keep a column and mention as advance."* So an advance is a balance reported beside the pay and never a term inside it. Netting the two would answer neither question — what the month earned, and what is still owed back — and a reader handed one merged figure can recover neither.

Person	Outstanding	Recovered so far
karim	65000	0
muskan	15000	0
vinay	5000	0

What holds all of this up

`python3 engine/tests/selftest.py` — 365 named checks over this engine, each one proven to fail before it was trusted to pass. The roster is compared name for name against the owner's own list; every stated leaving date is held to the day and to the month either side of it; the pay formula is checked against his own worked examples; and an advance is proven not to move the pay.

The product's 293 rules above and this engine are different things. A rule says what any business must refuse. This says what this one pays, and the numbers in it belong to the business rather than to the software.

Part four — what it runs on, and what would replace it

19 layers, 57 named alternatives between them. A layer with one option is a dependency; a layer with three is a choice. Every one names the interface everything above it talks to, which is what makes the swap possible rather than aspirational.

> **interface** — *A written promise about what a part of the system does, without saying which product does it — so the product underneath can be swapped without anything above noticing. Bijli ka socket. Socket ka size fix hai; usme koi bhi company ka plug lag jaata hai.* **adapter** — *A small piece of code that translates between the system and one outside service, so the rest of the system never has to know which service is in use. Travel adapter. Andar ka appliance wahi, bas plug ko us desk ke socket ke hisaab se badal diya.*

Layer	What it does	Built on	Alternatives	Everything talks to
The database	Keeps every record — customers, orders, stock, vouchers — and answers questions about them.	PostgreSQL	3	`DatabaseService`
File storage	Keeps photographs, invoices and scanned documents — the things too big to sit in the database.	Any S3-compatible object store	3	`FileStore`
Cache and short-term memory	Holds recently used answers and sign-in sessions so common screens open instantly.	Redis, or a Redis-compatible store	3	`CacheService`
The backend runtime	Runs the business rules, checks permissions, writes records and calculates totals.	Node.js with TypeScript	3	`the HTTP API contract`
The API	The agreed way the screens, the mobile view and any outside system ask the backend for things.	REST over HTTPS, with a written schema	3	`the published API schema`
The frontend	Everything a person sees and clicks — screens, forms, tables, dashboards.	React with TypeScript, screens generated from configuration	3	`the screen definition format`
Background work	Does the things nobody should have to wait for — sending a thousand messages, building a month-end report, pulling orders overnight.	A queue backed by the database, with named workers	3	`JobQueue`
Search	Finds a product, a customer or a document by a few typed letters, instantly.	PostgreSQL full-text search	3	`SearchService`
Sign-in and permissions	Proves who somebody is, then decides what they are allowed to see and change.	Sessions issued by the platform, with permissions checked in the backend and again in the database	3	`IdentityService`
Keys and passwords the system uses	Holds the connection details and keys the software needs, away from the code.	Environment variables on the server, readable only by the service account	3	`ConfigService`

Layer	What it does	Built on	Alternatives	Everything talks to
Messages to customers and staff	Sends WhatsApp messages, text messages and email — reminders, confirmations, statements.	A message service with one adapter per provider, per tenant	3	`MessageService`
Storefronts and marketplaces	Brings orders in from a shop website or a marketplace, and sends stock and prices back out.	A channel adapter per storefront or marketplace	3	`ChannelAdapter`
Taking payments	Collects money from customers online.	A payment adapter per provider, with the card field hosted by the provider	3	`PaymentService`
Delivery and couriers	Books a shipment, prints the label, and follows it to the door.	A courier adapter per carrier	3	`CourierService`
Artificial intelligence	Writes descriptions, tags photographs, summarises, and answers questions about your own data.	A router in front of several providers, ending on one that needs nothing bought	3	`ModelRouter`
Where it runs	The machines that serve the website and the application.	Containers on a virtual server	3	`the container image`
Source control and automatic checks	Keeps the history of every change and runs every test before anything goes live.	Git, with automatic checks on every change	3	`the test commands themselves`
Watching it	Reports errors, measures speed, and tells you when something stops answering.	Structured logs and error reporting, in an open format	3	`Logger and the metric format`
Making documents	Produces invoices, statements, labels and reports as files a person can print or send.	HTML templates printed to PDF by a headless browser	3	`DocumentRenderer`

The database

PostgreSQL is open source, runs anywhere, and has the two things this design needs built in: locks at the record level so one business cannot read another’s rows, and exact whole-number arithmetic so money never drifts. Any managed Postgres service is a hosting decision, not a database decision — the same schema runs on all of them.

- **Built on** — PostgreSQL
- **Could be replaced with** A managed Postgres service — same database, somebody else runs the machine
- Postgres on your own server — the software is free, you supply the machine
- MySQL or MariaDB, if a team already knows them well — the schema needs rework for the record-level locks
- **Everything talks to** — `DatabaseService`

- **Cost of switching** — Moving between Postgres hosts is a dump and a restore. Moving off Postgres entirely means rewriting the isolation layer, which is the one part worth not moving.

File storage

Almost every file service speaks the same request format, so one adapter reaches most of them. That makes this the cheapest layer in the whole system to change your mind about.

- **Built on** — Any S3-compatible object store
- **Could be replaced with** A different S3-compatible provider — usually a URL and a key change
- Files on your own server's disk, with a backup copy elsewhere
- A self-hosted object store such as MinIO, which speaks the same format
- **Everything talks to** — `FileStore`
- **Cost of switching** — Copy the files across and change the address. Nothing above this layer notices.

Cache and short-term memory

Nothing here is the only copy of anything. If the cache is wiped the system simply asks the database again and is a little slower for a minute — so this layer can be replaced, restarted or removed entirely without risking a single record.

- **Built on** — Redis, or a Redis-compatible store
- **Could be replaced with** Valkey — the open-source continuation of the same thing, same commands
- Memory inside the application itself, which is enough until traffic grows
- A database table, slower but with nothing extra to run
- **Everything talks to** — `CacheService`
- **Cost of switching** — Near zero by design. Losing the cache loses no data, which is the whole reason it is safe to change.

The backend runtime

The same language runs on the browser side, so one team can work across the whole system and code that validates a form can be shared with the code that validates the saved record — no rule gets written twice and no two versions of it drift apart.

- **Built on** — Node.js with TypeScript
- **Could be replaced with** Any container host — the code is ordinary and carries no host-specific parts
- Python or Go for a service that genuinely suits them, talking over the same API
- A different Node framework — the business logic sits outside the framework on purpose
- **Everything talks to** — `the HTTP API contract`
- **Cost of switching** — Low, because the rules live in plain functions rather than inside a framework. Moving a service means moving the functions and putting a different door in front of them.

The API

Ordinary web requests over predictable addresses. Anything can call it — a browser, a phone, a spreadsheet, another company's software — without a special library.

- **Built on** — REST over HTTPS, with a written schema
- **Could be replaced with** GraphQL for read-heavy screens, over the same underlying services
- A direct connection for live screens that must update by themselves
- Scheduled file exchange for partners who cannot call an API at all
- **Everything talks to** — `the published API schema`
- **Cost of switching** — Adding a second style is additive — the services underneath do not change.

The frontend

Screens are drawn FROM SETTINGS rather than written one by one. A tenant that renames a field, adds a column or turns a module off gets a different screen with no new code written — which is the only way one system can serve a steel plant and a single creator without becoming two systems.

- **Built on** — React with TypeScript, screens generated from configuration
- **Could be replaced with** Vue or Svelte — the screen definitions are plain data and do not care what draws them
- Server-rendered pages where speed on a weak connection matters more than interaction
- A native mobile shell reading the same screen definitions
- **Everything talks to** — `the screen definition format`
- **Cost of switching** — Moderate, and bounded: what a screen contains is data, so a rewrite replaces the painter, not the paintings.

Background work

Every job is written so that running it twice does the same thing as running it once. That single discipline is what makes it safe to retry after a failure, and it is worth more than any particular queue product.

- **Built on** — A queue backed by the database, with named workers
- **Could be replaced with** A Redis-backed queue when volume outgrows the database
- A hosted queue service, behind the same interface
- An external workflow tool such as n8n for steps a non-programmer should be able to edit
- **Everything talks to** — `JobQueue`
- **Cost of switching** — Low. Jobs are plain functions with a name; the queue only decides when they run.

Search

Postgres can search well enough for a long time, and starting there means one less thing running, one less thing to back up, and one less thing to keep in step with the database.

- **Built on** — PostgreSQL full-text search

- **Could be replaced with** OpenSearch or Elasticsearch when catalogues grow large
- Meilisearch or Typesense — small, fast, self-hostable
- A hosted search service behind the same interface
- **Everything talks to** — `SearchService`
- **Cost of switching** — Low, and it is a one-way door you can walk back through: the records stay in the database either way, so a search engine is only ever a faster copy.

Sign-in and permissions

Who you are and what you may do are kept apart deliberately. Sign-in can be handed to an outside service — or to a customer's own company login — while permissions stay ours, because they depend on the company and role structure no outside service knows about.

- **Built on** — Sessions issued by the platform, with permissions checked in the backend and again in the database
- **Could be replaced with** An identity provider for sign-in only, with permissions still decided here
- A customer's own company sign-in, for enterprises that require it
- Self-hosted Keycloak or Authentik, when nothing may leave the building
- **Everything talks to** — `IdentityService`
- **Cost of switching** — Low for sign-in, by design. Permissions never move, so the expensive half is never in play.

Keys and passwords the system uses

A key in the code is a key in every copy of the code forever. Keeping them outside means one can be replaced in a minute without changing a line.

- **Built on** — Environment variables on the server, readable only by the service account
- **Could be replaced with** A managed secrets service, when there are enough of them to be worth it
- Self-hosted Vault or Infisical
- Encrypted files kept outside source control
- **Everything talks to** — `ConfigService`
- **Cost of switching** — Very low — the code asks for a name and does not care where the value came from.

Messages to customers and staff

Each tenant connects its own accounts. The platform is built with a place for them to plug in and never holds one central account of its own — a business's conversations with its own customers belong to that business. The platform's job is the plug, not the account.

- **Built on** — A message service with one adapter per provider, per tenant
- **Could be replaced with** Any WhatsApp provider — the adapter changes, the code that decides what to send does not
- Text message and email as fallbacks when a message cannot be delivered

- A shared inbox or an export, for a tenant with no messaging account at all
- **Everything talks to** — `MessageService`
- **Cost of switching** — One adapter per provider. Switching is a settings change made by the tenant, not a release made by us.

Storefronts and marketplaces

Every one of these is treated as a channel with an adapter. Adding a marketplace is writing one adapter and creating one record — never a change to how orders work.

- **Built on** — A channel adapter per storefront or marketplace
- **Could be replaced with** A different storefront platform — a new adapter, and orders keep arriving
- File import for a channel with no connection available
- Manual entry, which must always remain possible
- **Everything talks to** — `ChannelAdapter`
- **Cost of switching** — One adapter each. The order, the stock number and the books never change shape.

Taking payments

Card details are handed straight to the payment provider's own secured field and never touch this system — so there is nothing sensitive here to protect, and switching provider moves no card data, because none was ever held.

- **Built on** — A payment adapter per provider, with the card field hosted by the provider
- **Could be replaced with** Any other payment provider, behind the same interface
- Bank transfer and UPI details recorded against the invoice
- Cash on delivery, reconciled when the courier settles
- **Everything talks to** — `PaymentService`
- **Cost of switching** — One adapter. No card data ever moves, because none is ever stored.

Delivery and couriers

Rate cards and tracking differ per courier; what a shipment IS does not.

- **Built on** — A courier adapter per carrier
- **Could be replaced with** A courier aggregator, which is itself just one more adapter
- A different carrier directly
- Manual booking with the tracking number typed in — always available
- **Everything talks to** — `CourierService`
- **Cost of switching** — One adapter each.

Artificial intelligence

Ordered fallback, a breaker on anything failing repeatedly, and a spend ceiling that REFUSES rather than warns. Because every capability also has an option that costs nothing, a spent budget can stop the spending without ever

stopping the business.

- **Built on** — A router in front of several providers, ending on one that needs nothing bought
- **Could be replaced with** Any hosted model provider — an entry in the router, not a change to the system
- A model running on your own machine, for work that is routine or private
- Templates and rules with no model at all, which must always remain the last resort
- **Everything talks to** — `ModelRouter`
- **Cost of switching** — A list entry. The router exists precisely so changing provider is never a project.

Where it runs

The application is packaged as an ordinary container with nothing host-specific inside it. That single decision is what keeps every hosting option open, forever.

- **Built on** — Containers on a virtual server
- **Could be replaced with** A managed container platform, when scaling by hand stops being fun
- A different cloud, or a different country, for the same container
- A machine in your own office, for data that must not leave it
- **Everything talks to** — `the container image`
- **Cost of switching** — Low by construction. If moving hosts is ever hard, something host-specific has leaked in and that is the bug.

Source control and automatic checks

Git itself is the thing that matters, and git is not owned by anybody. The host is a convenience.

- **Built on** — Git, with automatic checks on every change
- **Could be replaced with** A different hosting service — a git repository moves with one command
- Self-hosted Gitea or Forgejo
- A separate build service reading the same repository
- **Everything talks to** — `the test commands themselves`
- **Cost of switching** — Very low. The checks are ordinary commands, so any system that can run a command can run them.

Watching it

Standard formats mean the tool that reads them is replaceable without changing what the system emits.

- **Built on** — Structured logs and error reporting, in an open format
- **Could be replaced with** Any hosted error-tracking service
- Self-hosted GlitchTip, or a Grafana and Prometheus stack
- Log files plus an uptime checker, which is enough at the start
- **Everything talks to** — `Logger and the metric format`
- **Cost of switching** — Low — the system emits a standard shape and does not know who is reading it.

Making documents

What a document SAYS is data. How it is drawn is replaceable, and should be.

- **Built on** — HTML templates printed to PDF by a headless browser
- **Could be replaced with** A dedicated PDF library for very high volume
- A hosted document service
- Spreadsheet or CSV output, which some readers prefer anyway
- **Everything talks to** — `DocumentRenderer`
- **Cost of switching** — Low. Templates are content; the renderer is a tool.

Part five — what changes without a developer, and what never changes

> **effective date** — *The date a change starts applying from. Records made before it keep the old value; records after it use the new one.* *Naya rate 1 tarikh se lagu. Purane mahine ka bill purane rate se hi banega — woh apne aap nahin badlega.*

18 things you can change, across 4 areas — and 6 that can never be switched off. Everything below is changed in the app, by you, taking effect the same minute. None of it needs a developer, a release or a phone call.

The column that matters most is the last one: **what happens to records already made.** A change carries the date it starts from and is added rather than written over, so a supervisor can leave on Tuesday and a replacement start on Wednesday — and last month's payroll, already paid, does not move by a rupee. *Purana record mitta nahin; naye date se naya rule lagta hai.*

People

What changes	Who can	Takes effect	Records already made
Somebody joins — a worker, a supervisor, an office staff member, a contractor	Admin, or an HR role	They exist from their start date and can be assigned work the same minute.	Nothing before their start date mentions them, because they were not there.
Somebody leaves, with or without notice	Admin, or an HR role	Marked as left from a date. Their sign-in stops, and no new work is assigned to them. Their record is kept, not deleted.	Every hour they worked, every piece they made and every rupee they were paid stays exactly as it was. Deleting the person would blank all of it and change months already closed — so the record remains and simply has an end date.
A replacement starts immediately, in the same position	Admin, or an HR role	Added with their own start date and given the position. Work in progress is reassigned to them from that date. No waiting, no release, no developer.	Work completed under the previous person stays credited to the previous person. Two people held the same position at different times, and every record knows which one applied when.
A pay rate, a piece rate or a salary changes	Admin, or an HR role	Applies from the date you set — which may be today, a future date, or a past one you are correcting.	Every completed period recalculates to the rate that applied then, not the new one. This is the single most important line in this register: a rate that silently applied backwards would change payments already made to real people.
What somebody is allowed to see and do	Admin	Takes effect on their next action. Screens they may no longer open stop opening.	Everything they did while they held the old permissions stays recorded, with the permissions they had at the time.

Structure

What changes	Who can	Takes effect	Records already made
A new company is opened, or an existing one is closed	Admin	It exists immediately with its own name, trading name, code and document numbering. The group view includes it from that date.	Group figures for earlier periods are unchanged, because the company did not exist in them.
A new way of selling is added — a marketplace, a shop, a counter, an export desk	Admin	Orders can arrive through it the same day. It appears in every report that breaks figures down by channel.	Earlier reports keep their own columns. A channel that did not exist then does not appear then.
A godown, a shop, a unit or a stock point is added, renamed or closed	Admin	Stock can move to and from it immediately.	Stock movements already recorded keep pointing at it, under the name it had at the time.
Which parts of the system this business uses at all	Admin	Turned on, a module appears in the menu with its screens ready. Turned off, it disappears from the menu. A steel plant, a clothing brand and a single creator each end up with a different system built from identical code.	Turning a module off hides its screens and keeps its records. Nothing is destroyed by tidying a menu.

Your words

What changes	Who can	Takes effect	Records already made
What the system calls things	Admin	Every screen, every report and every document changes wording at once. One business says order, another says job, another says matter, consignment, batch or booking — the record underneath is identical.	Documents already issued keep the wording they were issued with, because that is what the customer received.
Extra information you want to record that nobody else needs	Admin	Added to the screen immediately, with the type you choose — text, number, date, a list to pick from, a yes or no. Reportable from the moment it exists.	Older records simply have no value for it, which is the truth. They are never back-filled with a guess.
The steps your work moves through	Admin	Add a stage, rename one, reorder them or remove one. New work follows the new list from that moment.	Work already part-way through keeps the stage it is in, even if that stage has since been removed — a job does not teleport because somebody edited a list.
The layout and numbering of invoices, statements, labels and reports	Admin	The next document uses the new layout or the new numbering.	Documents already issued are never re-rendered. What the customer holds and what you hold stay identical.

Rules

What changes	Who can	Takes effect	Records already made
Turning a discretionary rule on or off	Admin	Applies to the next transaction. One business requires an approval below a price floor; another does not — same software, different setting.	Transactions already posted are not re-judged against a rule that did not apply to them.
Who has to approve what, and above which amount	Admin	The next request follows the new path.	Requests already approved keep the path they went through, and the names of who approved them.
Tax rates and the categories they attach to	Admin, or an accounts role	Applies from its effective date, which for tax is set by law rather than by you.	Every invoice keeps the rate that applied on its own date. A return filed for an earlier period recalculates to that period's rate — this is not a convenience, it is the only correct behaviour.
Which outside service is used for messages, payments, delivery or artificial intelligence	Admin	The next message, payment or shipment goes through the new one.	Everything already sent keeps the record of which service carried it, which is what you need when you query one.
The most the system may spend on paid outside services	Admin	Enforced immediately. Over the ceiling, the paid option is refused and the work completes on one that costs nothing.	Spending already recorded is unchanged.

What can never be switched off

Short on purpose. Every line is something a bank, an auditor, a customer or an employee relies on, and a setting that could remove it would remove their protection too.

Never changeable	Why
The audit trail	Who changed what, and when. A system where this can be switched off cannot be used to answer a dispute, so it cannot be switched off.
Every record naming the company it belongs to	Without it, figures from two companies merge and no report can be trusted again.
One business being unable to read another’s records	This is not a preference. It is the promise that makes a shared platform usable at all.
Money kept as exact whole units	The alternative loses fractions of a rupee in ways nobody can trace afterwards.
Deleting nothing — records are ended, never erased	An erased record changes a period that was already closed, filed and possibly audited.
Never asking for a marketplace, bank or account password	The system connects through proper keys that you can withdraw. A password would hand over an account you cannot take back.

Every technical word in this document, in plain language

- **platform** — One piece of software that many separate businesses use at the same time, each seeing only its own information. *Ek badi building jisme bahut saare offices hain. Building ek hai, par har office ki chaabi alag — koi kisi aur ke office mein nahin ghus sakta.*
- **tenant** — One business using the platform. Its people, its data and its settings are its own. *Us building mein ek office. Aapka office, aapka saamaan, aapka taala.*
- **industry pack** — A settings file that teaches the system your trade — what you call things, the stages your work moves through, the documents you issue. *Ek hi machine, alag-alag saancha. Saancha badal do, wahi machine doosri cheez banane lagti hai.*
- **database** — Where all the information is kept, arranged so any of it can be found instantly and nothing gets lost. *Ek badi almari jisme har cheez apne fix khaane mein rakhi hai — dhoondhne ke liye poori almari palatni nahin padti.*
- **schema** — The written plan of what information the system keeps and how the pieces connect. *Makaan ka naksha. Deewar uthane se pehle kaagaz pe tay hota hai kaunsa kamra kahaan hai.*
- **row-level security** — A lock inside the database itself, so one business physically cannot read another business’s records — even if the software above it has a bug. *Taala darwaze pe nahin, tijori pe. Guard so bhi jaaye toh bhi tijori band rehti hai.*
- **migration** — A recorded change to the shape of the database, so every copy of the system can be updated the same way, in the same order. *Naksha badla toh likh ke rakha — taaki har site pe wahi badlav, usi tarike se ho.*
- **backup** — A copy of everything, kept somewhere else, so a mistake or a failure does not lose your work. *Zaroori kaagzaat ki photocopy, doosri jagah rakhi hui. Asli jal jaaye toh bhi kaam nahin rukta.*

- **cutover** — The moment the business stops using the old way of working and starts using the new one for real. *Woh din jab purana tarika band aur naya shuru — ab asli kaam nayi jagah pe hoga.*
- **backend** — The part of the software you never see, which does the actual work — checks the rules, saves the records, calculates the totals. *Hotel ka kitchen. Customer nahin dekhta, par khaana wahin banta hai.*
- **frontend** — The part you see and click — the screens, the buttons, the forms. *Hotel ka dining hall aur menu card. Jo aapke saamne hai.*
- **API** — The agreed way two pieces of software talk to each other, so one can ask the other for something and get a predictable answer. *Waiter. Aap kitchen mein nahin jaate — waiter ko order dete ho, wahi khaana le aata hai. Waiter badal jaaye toh bhi order dene ka tarika wahi rehta hai.*
- **storage** — Where files are kept — photographs, invoices, scanned documents. Different from the database, which keeps information rather than files. *Almari ke bagal wala godown. Register almari mein, par bade dabbe aur photo godown mein.*
- **queue** — A waiting line for work that does not have to finish this second — sending a hundred messages, building a big report. *Darzi ki dukaan ka parchi system. Kaam parchi pe likh ke lag gaya line mein; customer khada intezaar nahin karta.*
- **job** — One piece of work taken off the queue and done in the background. *Line mein se uthayi gayi ek parchi, ab uska kaam ho raha hai.*
- **environment** — A separate running copy of the system — one for trying things, one that customers actually use. *Rehearsal aur asli show. Practice alag jagah, taaki galti sabke saamne na ho.*
- **deployment** — Putting a new version of the software in place so people start using it. *Nayi dukaan kholna ya purani ko naya roop dena — jab tak shutter nahin uthta, customer ko farq nahin padta.*
- **rollback** — Putting the previous working version back, quickly, when a new one turns out to be wrong. *Naya taala kharab nikla toh purana taala wapas laga do — do minute ka kaam.*
- **uptime** — How much of the time the system is actually working and reachable. *Dukaan mahine mein kitne din khuli rahi. Band rahi toh customer wapas chala gaya.*
- **model** — The piece of artificial intelligence that reads or writes text, tags a photograph, or answers a question. *Ek bahut padha-likha assistant. Kaam accha karta hai, par har baat pe usse poochho toh kharcha aur waqt dono lagta hai.*
- **provider** — A company whose service the system uses — for messages, for payments, for artificial intelligence, for delivery. *Supplier. Ek supplier maal na de toh doosre se le lo — kaam nahin rukna chahiye.*
- **fallback** — The next option the system automatically moves to when the first one fails or is unavailable. *Bijli gayi toh inverter. Inverter gaya toh mombatti. Andhera kabhi nahin hota.*
- **spend ceiling** — A maximum amount the system is allowed to spend on paid services, after which it refuses to spend more instead of warning you. *Jeb mein utne hi paise leke nikle jitna kharch karna hai. Khatam matlab khatam — udhaar nahin.*
- **role** — What a person is allowed to see and do — a manager sees more than a counter staff member. *Chaabi ka guccha. Manager ke paas zyada chaabiyaan, staff ke paas kam.*
- **permission** — One specific thing a role is allowed to do, like approving a discount or viewing salaries. *Guchhe ki ek chaabi. Ek chaabi ek darwaza.*